

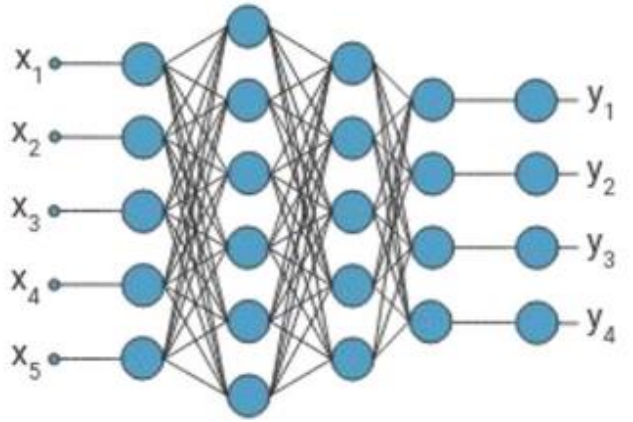
TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

INTELIGENCIA ARTIFICIAL

Prof. Me. Matheus Raffael Simon

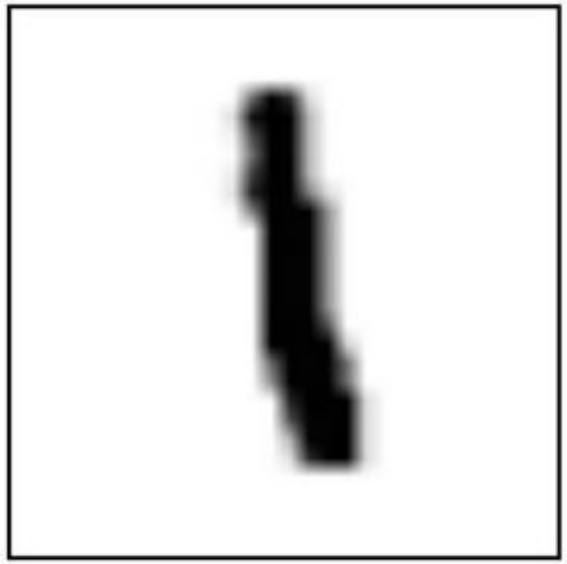
Fully Connected Artificial Neural Nerworks

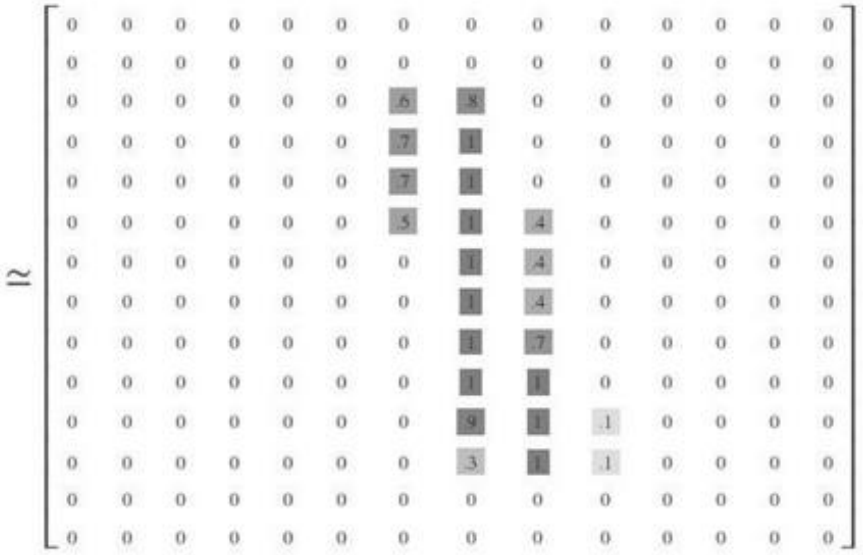
- Redes Neurais Artificiais Totalmente Conectadas (Fully Connected) não são muito adequadas para classificação de imagens
- grande quantidade de parâmetros > **overfitting**
- difícil lidar com imagens de alta resolução
- Uma simples camada pode ter mais de 1 bilhão de parâmetros!
- computacionalmente “caras” > tempo exponencial para computar ativações



FC - ANNs

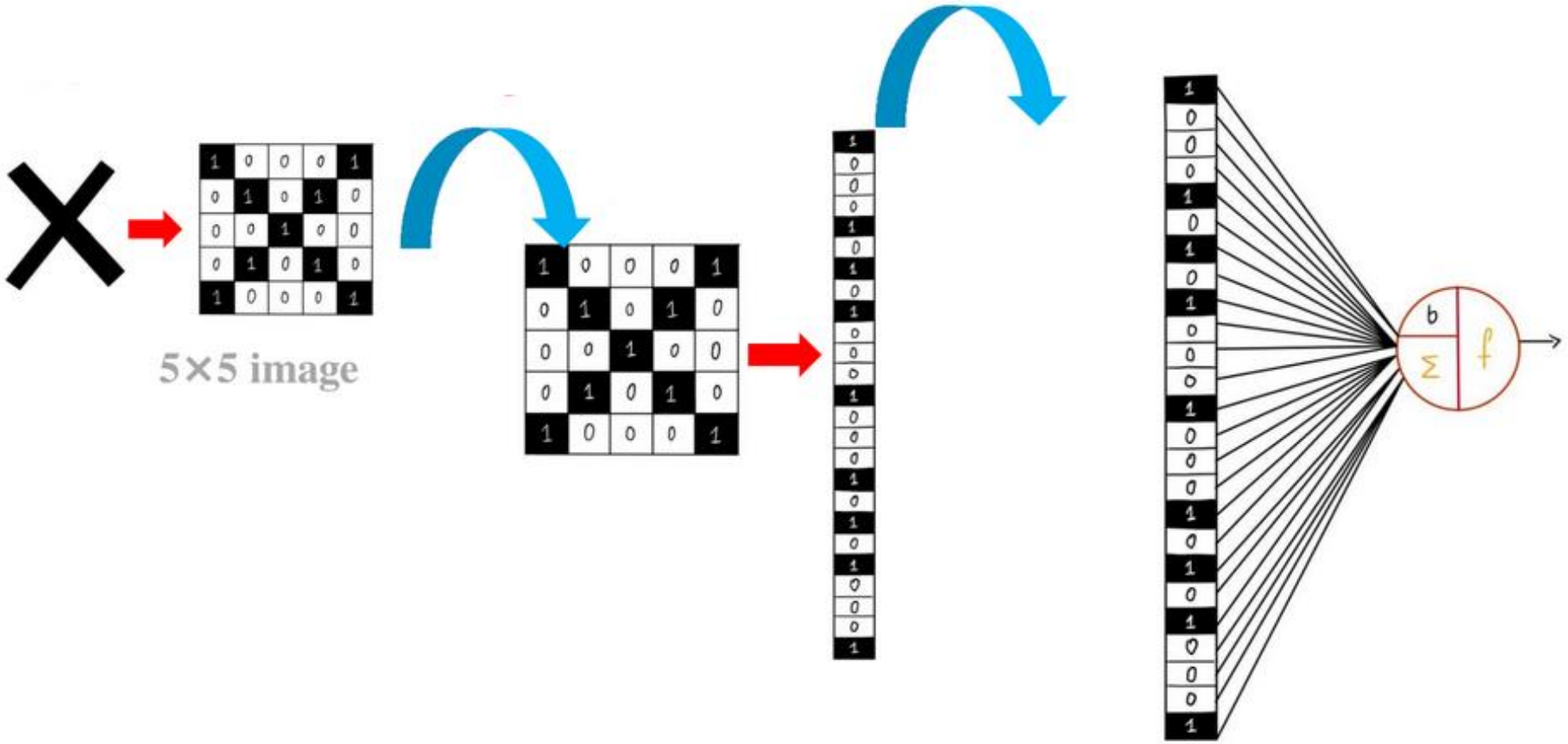
- **FC-ANNs ignoram** a estrutura espacial existente em imagens
- **imagens linearizadas >** pixels vizinhos e os localizados em regiões distantes tratados indistintamente
- idealmente a própria ANN teria que **detectar as dependências existentes na estrutura espacial** da distribuição subjacente às imagens de entrada





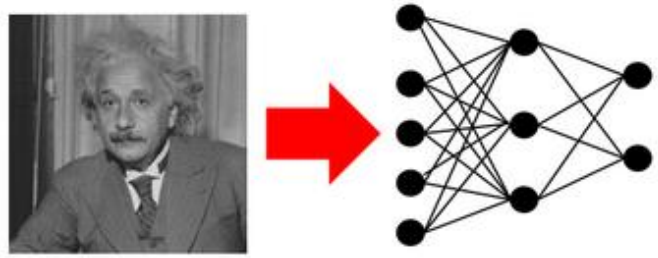
FC - ANNs

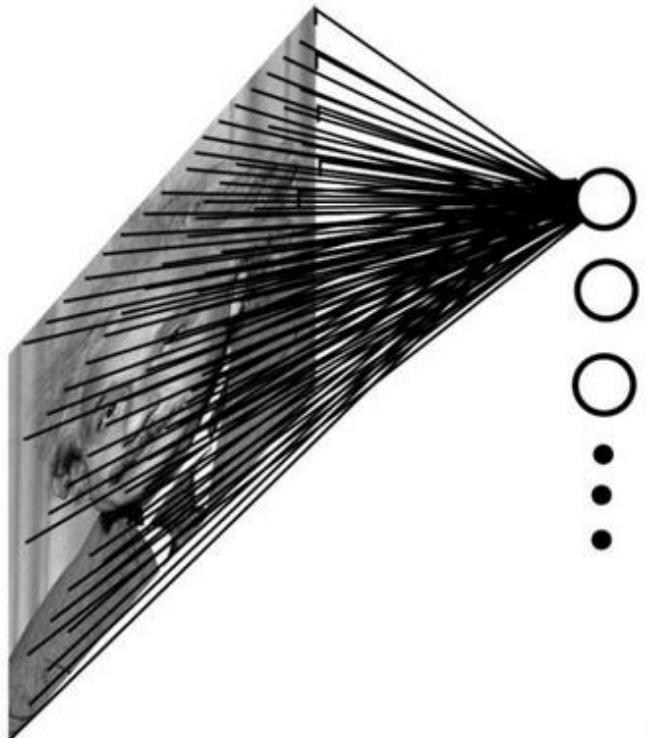
- **Um único neurônio**



FC - ANNs

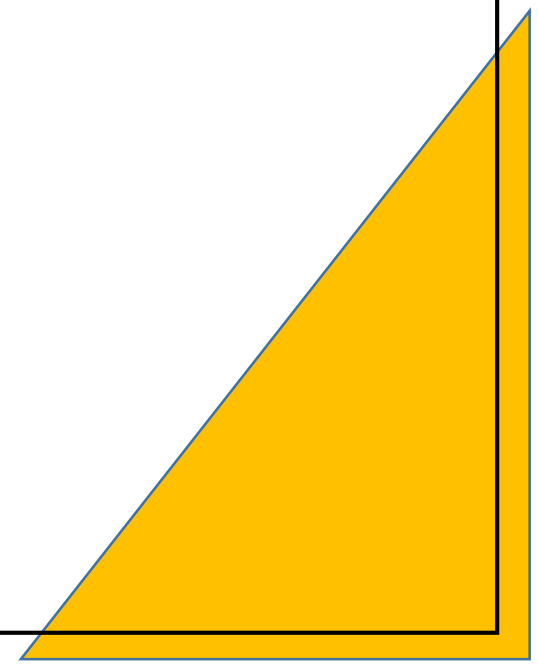
- Exemplo
- imagem 100 × 100 pixels
- Mil neurônios de entrada
- 10^7 conexões!
- Correlação espacial é perdida!
- Entradas de **dimensão variável**





FC - ANNs

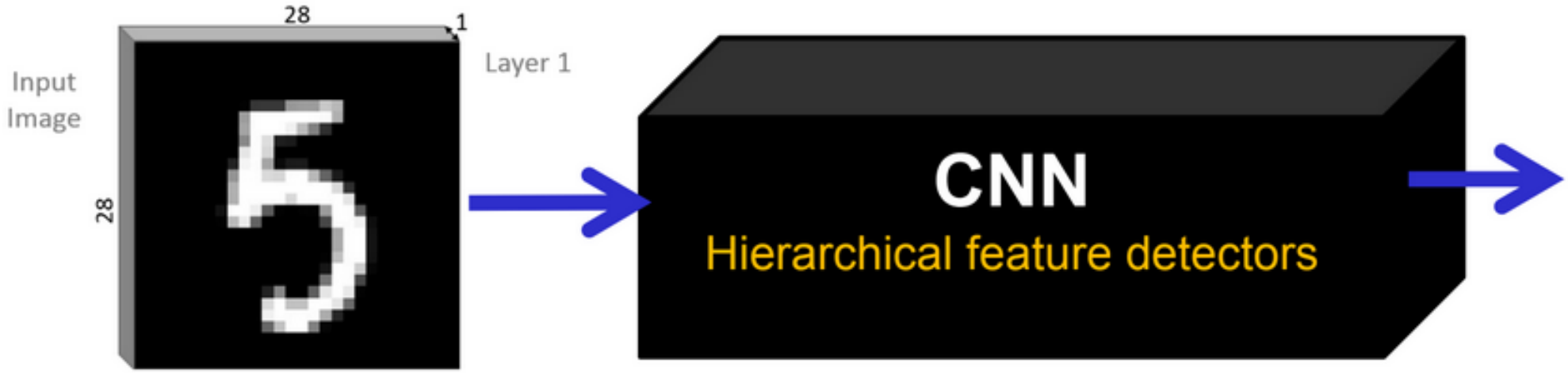
- Exemplo
- imagem 100 × 100 pixels
- Mil neurônios de entrada
- 10^7 conexões!
- Correlação espacial é perdida!
- Entradas de **dimensão variável**




Who
will
save US?

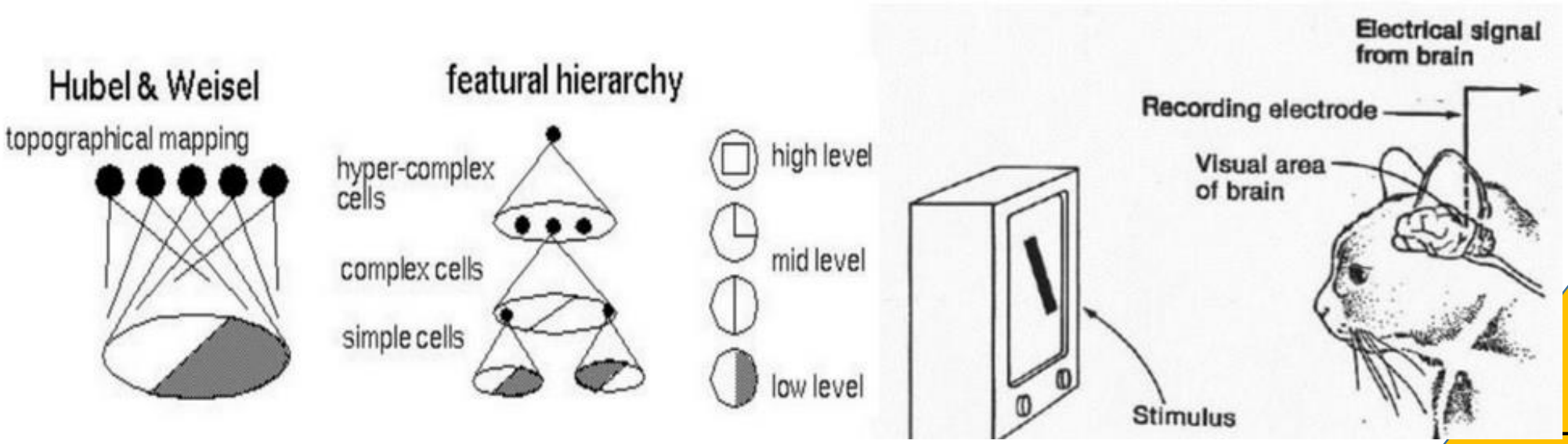
Convolutional Neural Network (CNN)

- Uma Convolutional Neural Network (CNN) é adaptada para explorar a correlação espacial existente em imagens
- Inspirada no funcionamento do córtex visual do cérebro de mamíferos



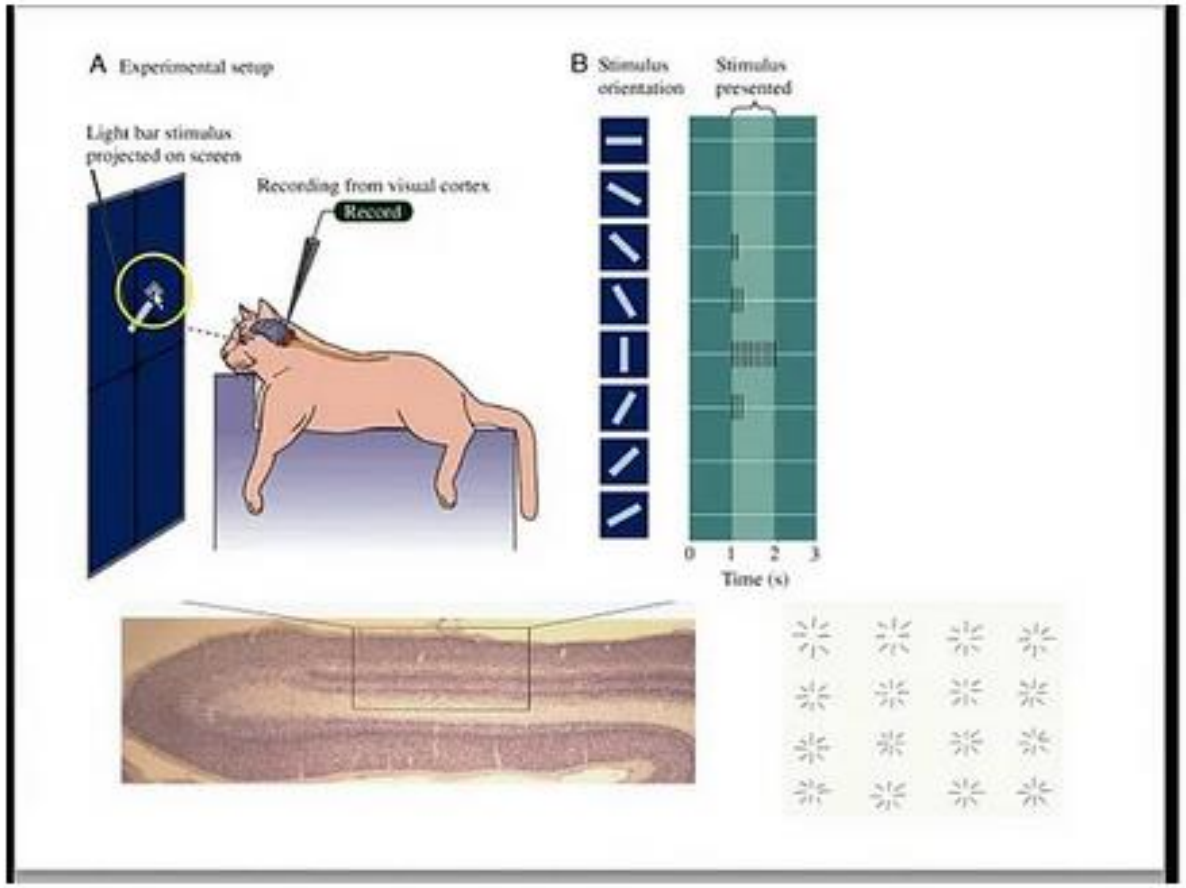
Inspiração Biológica: Córtex Visual Mamífero

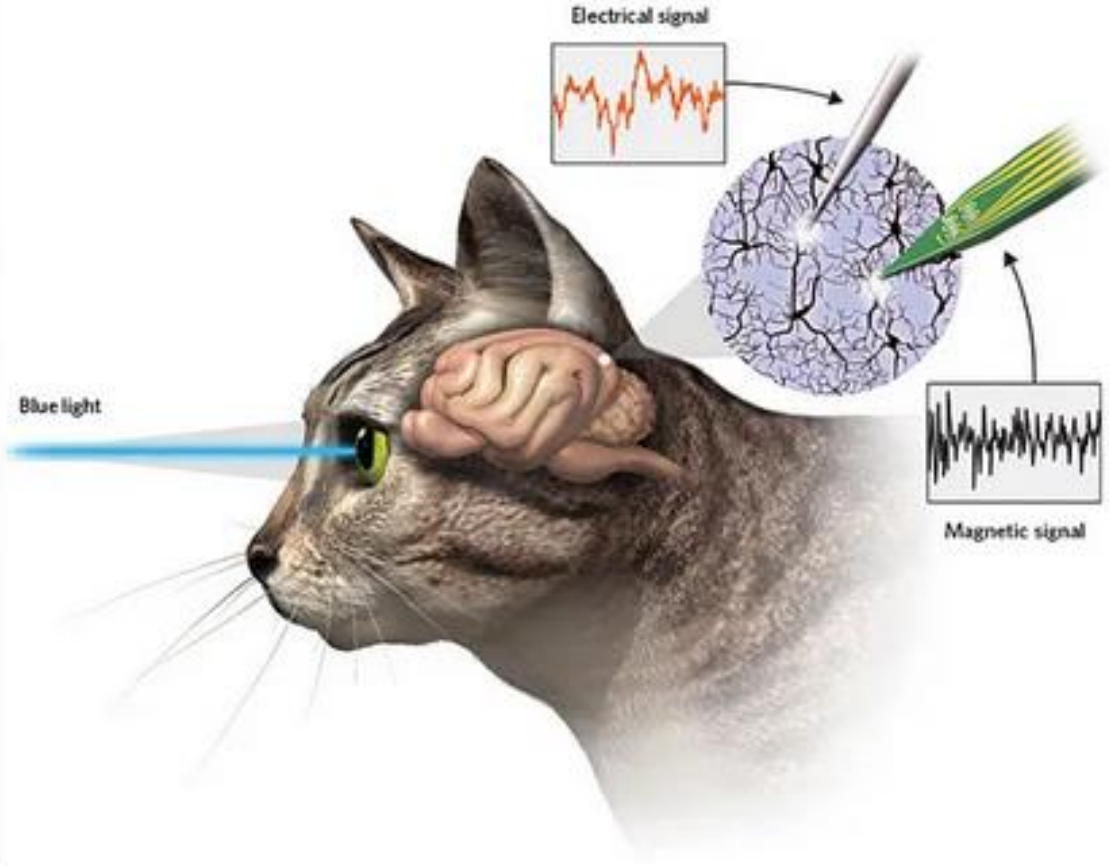
- Arquitetura em camadas de Hubel & Weisel (1959) > percepção visual



Inspiração Biológica: Córtex Visual Mamífero

- https://medium.com/@adityajilla126/cnn-vs-visual-cortex-the-cat-experiment-that-changed-ai-forever-5c61da74aa31 Acesso 20/05/2025

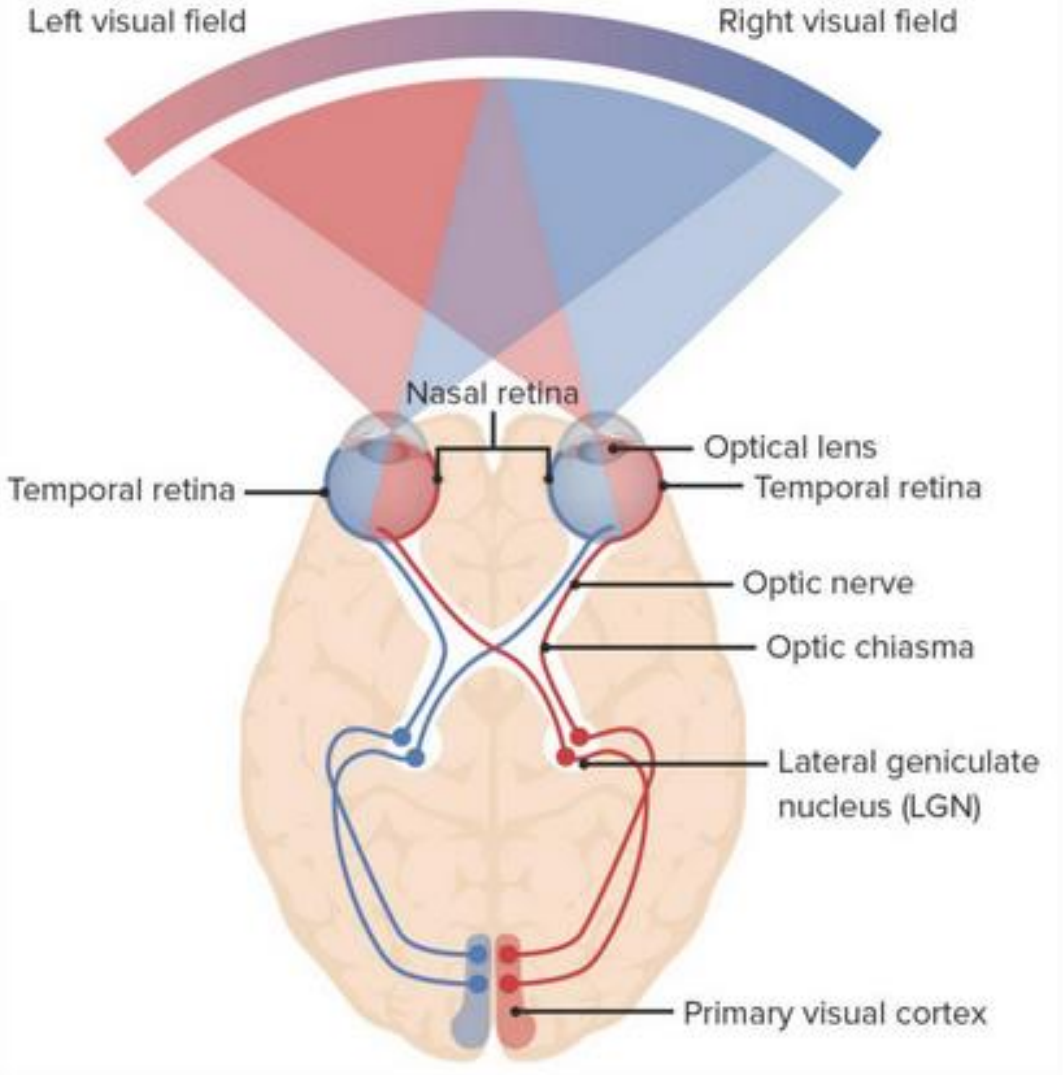


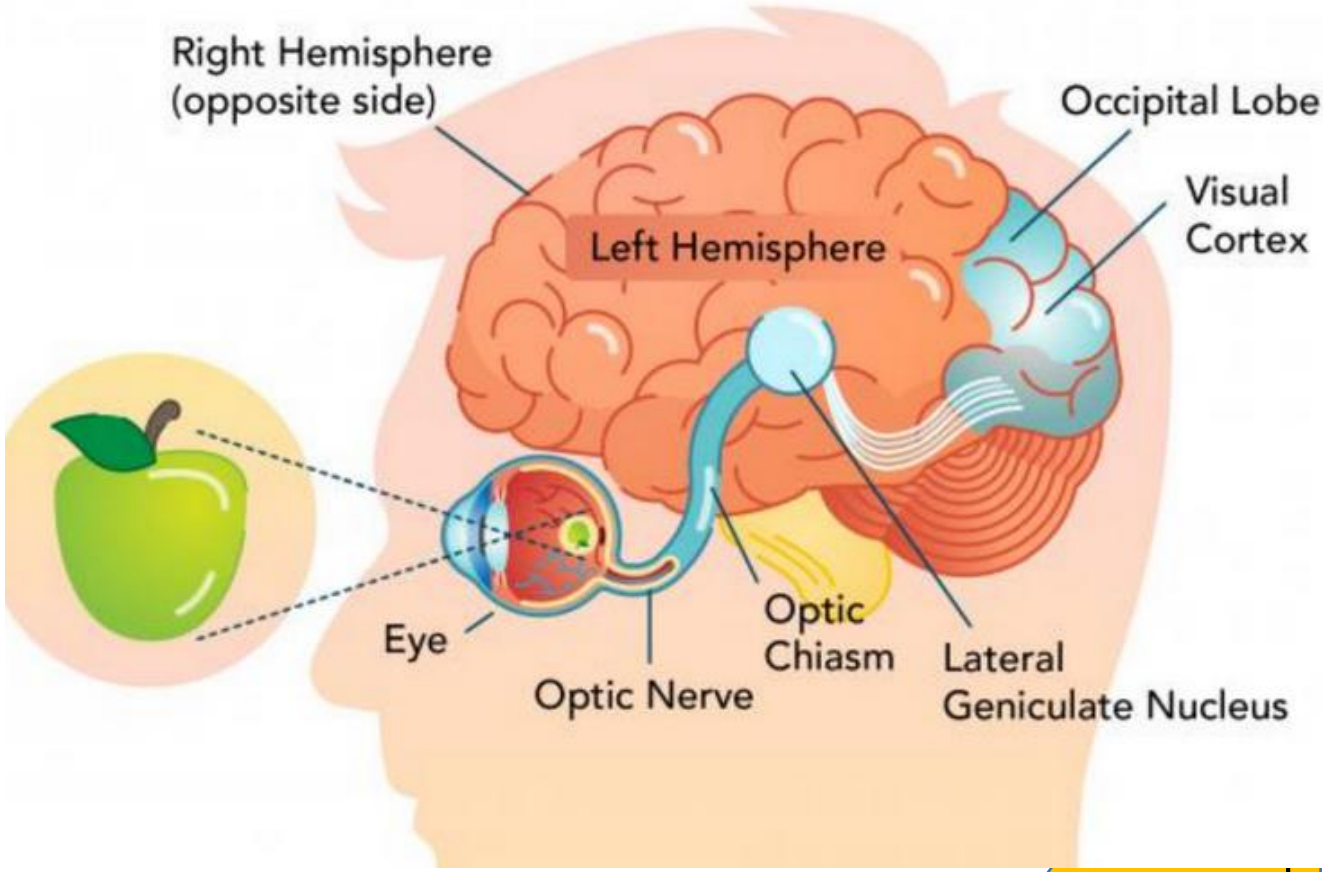


Inspiração Biológica: Córtex Visual Mamífero

- O córtex visual é uma região do cérebro responsável pela informação visual, incluindo movimento e tridimensionalidade
- Área V1 (primário) > forma, orientação e movimento de objetos
- Outras áreas > informações mais complexas como cor, textura e identidade de objetos

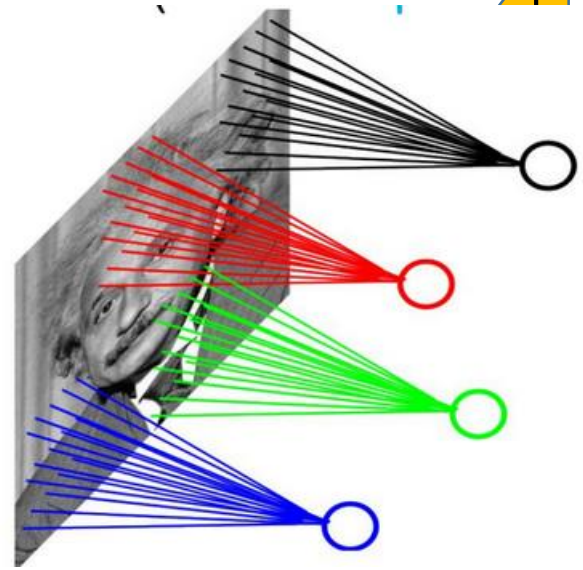
Inspiração Biológica: Córtex Visual Mamífero





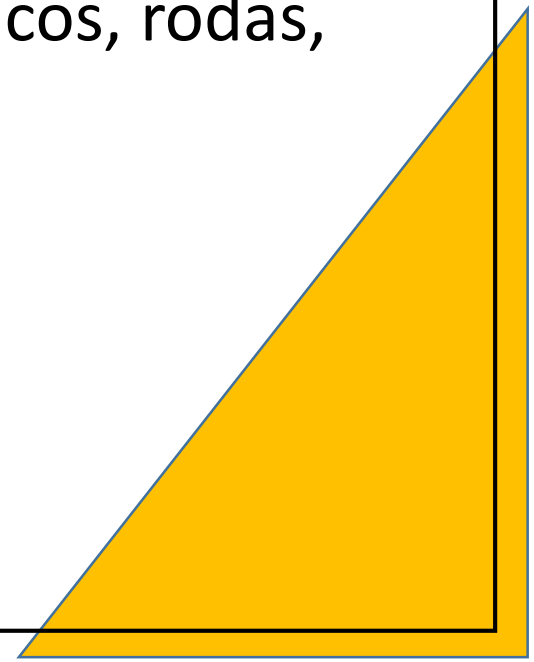
CNN: campo receptivo local

- Neurônios em uma CNN utilizam **conectividade local** e não conectividade global
- cada neurônio está conectado a uma **região contígua** de unidades da camada anterior
- Essa região é o **campo receptivo (local receptive field)** dessa unidade



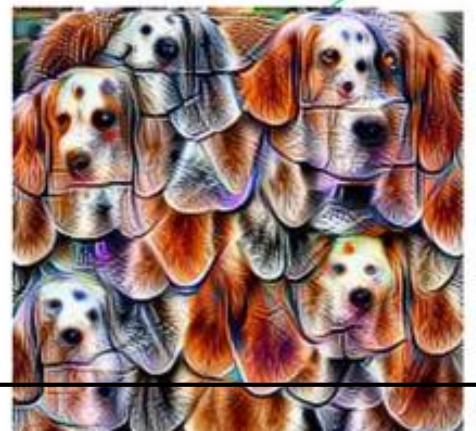
CNN: campo receptivo local

- Por meio de seu campo receptivo, cada neurônio pode detectar **características visuais** elementares > arestas orientadas, extremidades, cantos, etc.
- podem então ser combinadas por camadas subsequentes para detectar **características visuais mais complexas** > olhos, bicos, rodas, etc.



CNN: campo receptivo local

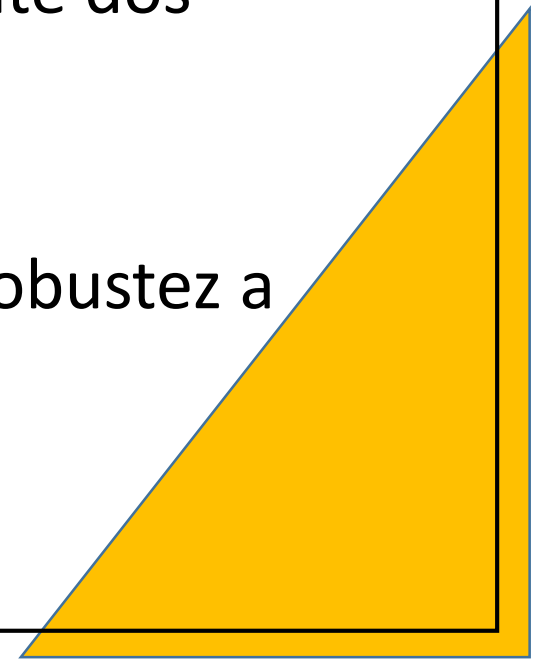




CNN

- CNNs são similares a outras NNs
- são treinadas com uma versão de algoritmo **back-propagation**
- Difere a arquitetura delas:
- São projetadas para reconhecer padrões visuais diretamente dos pixels com nenhum (mínimo) preprocessamento

- Podem reconhecer padrões com extrema variabilidade e robustez a ruídos, distorções e transformações geométricas simples
- Ex.: caracteres manuscritos

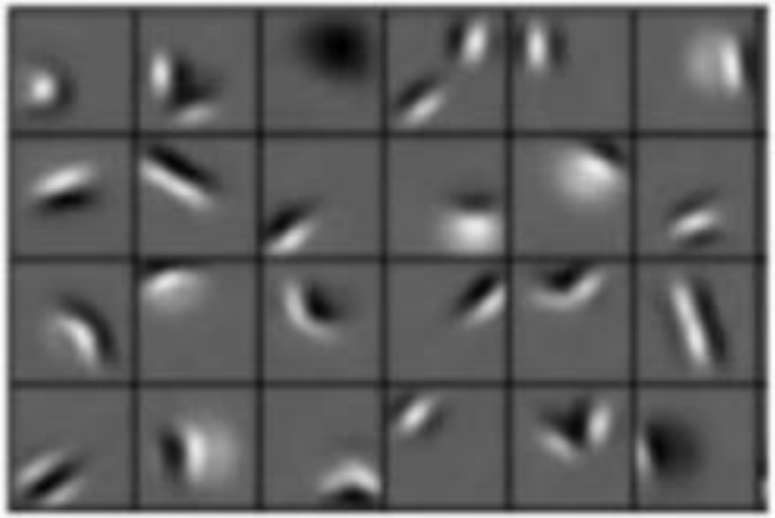


CNN

- CNN é uma hierarquia de camadas (layers)
- Cada camada transforma os dados de entrada em representações mais abstratas
- Ex.: contornos > nariz > face
- Camada final combina as representações > **Predição**

CNN

Low level features



Edges, dark spots

Mid level features



Eyes, ears, nose

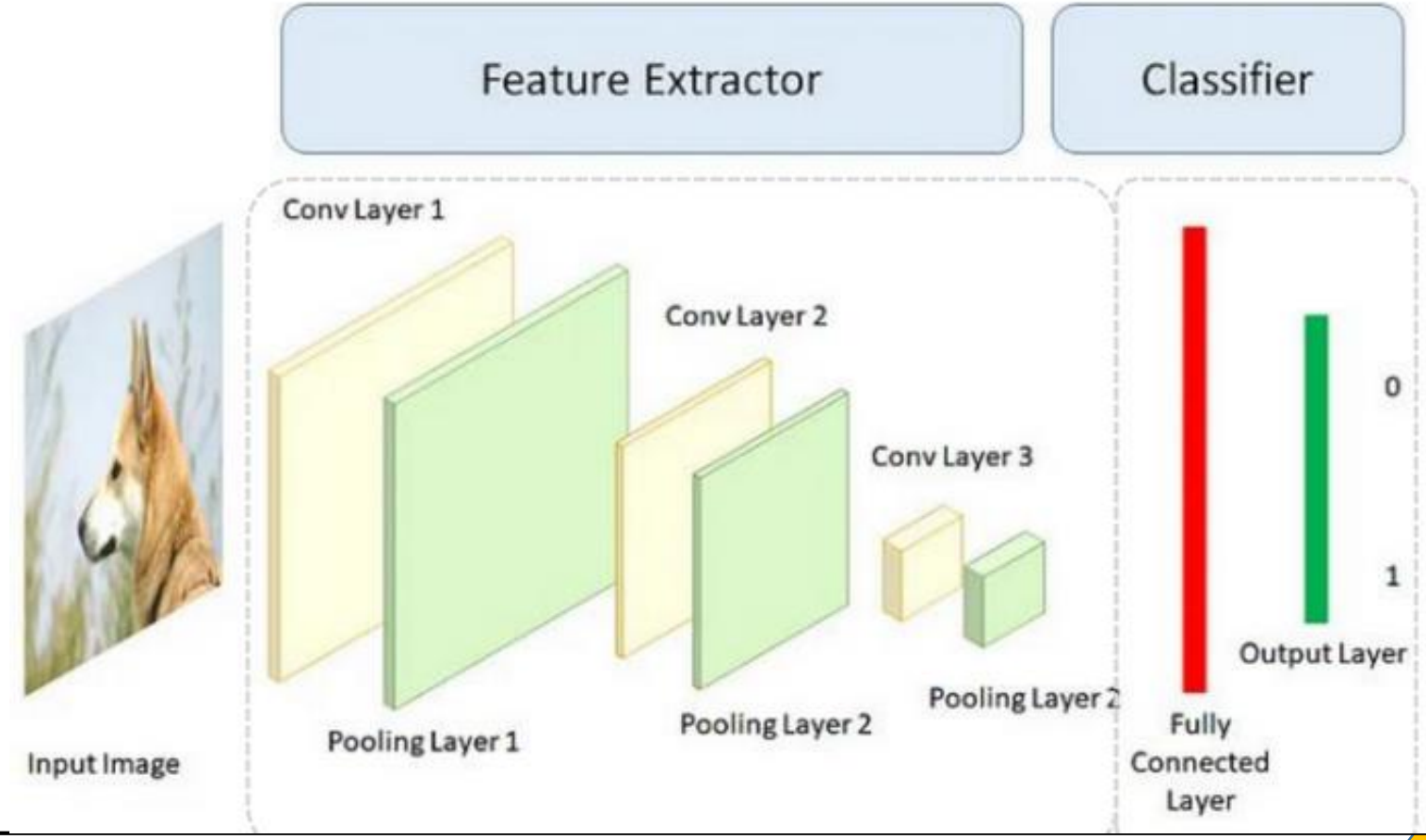
High level features



Facial structure

CNN: Esquema geral

- Resumindo > temos um **feature extractor** seguido de um **classifier**

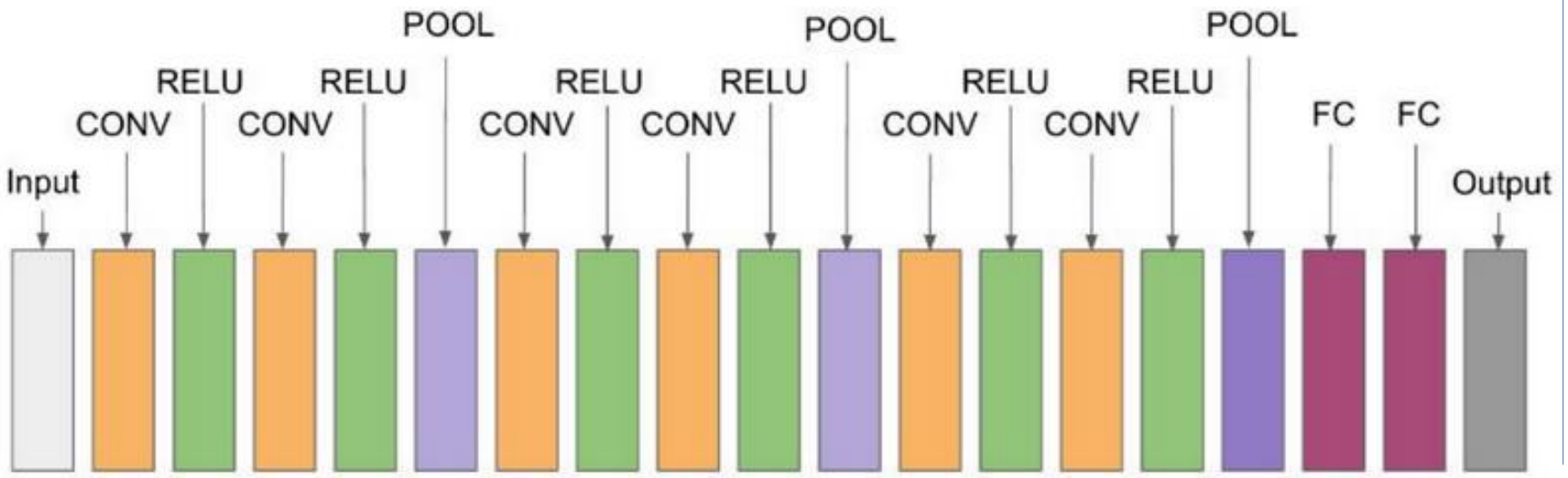


Camadas de uma CNN



CNN: Camadas

- Uma CNN é essencialmente um arranjo de camadas (layers):
- Convolutional Layers
- ReLU (Rectfied Linear Units Layers)
- Pooling Layers
- Fully-Connected Layers



CNN: Convolução

- Mas que Censurado é uma Convolução, não?



Convolução espacial

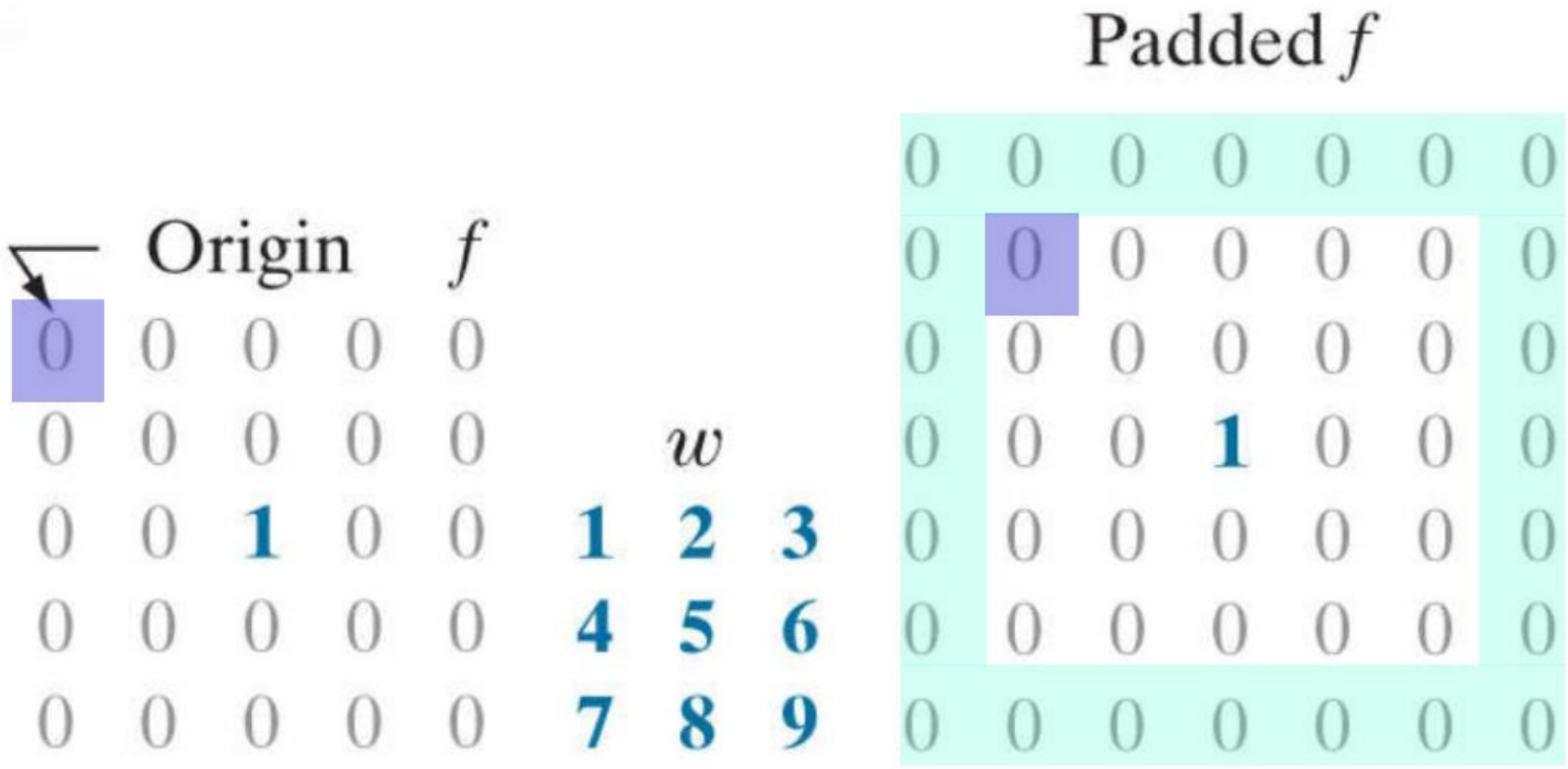
- A convolução espacial de uma imagem $f_{M \times N}(x, y)$ com kernel $w_{m \times n}$ é dada por:

$$(w \star f)(x, y) = \sum_{s=-a}^{a} \sum_{t=-b}^{b} w(s, t) f(x - s, y - t)$$

- **o centro do kernel w(0, 0) alinha-se com o pixel (x, y) de f e visita**
- **cada pixel de f uma vez e o kernel é rotacionado 180o**
- **assumindo f preenchida (padded)**
- **a = (m -1) / 2 e b = (n - 1) / 2, m e n inteiros ímpares**

Convolução espacial: zero padding

- Função 2-D *f* e *kernel* w_{3x3}



Convolução espacial: zero padding

- **Convolution kernel w_{3x3}**

1	-1	-1
1	**2**	-1
1	1	1

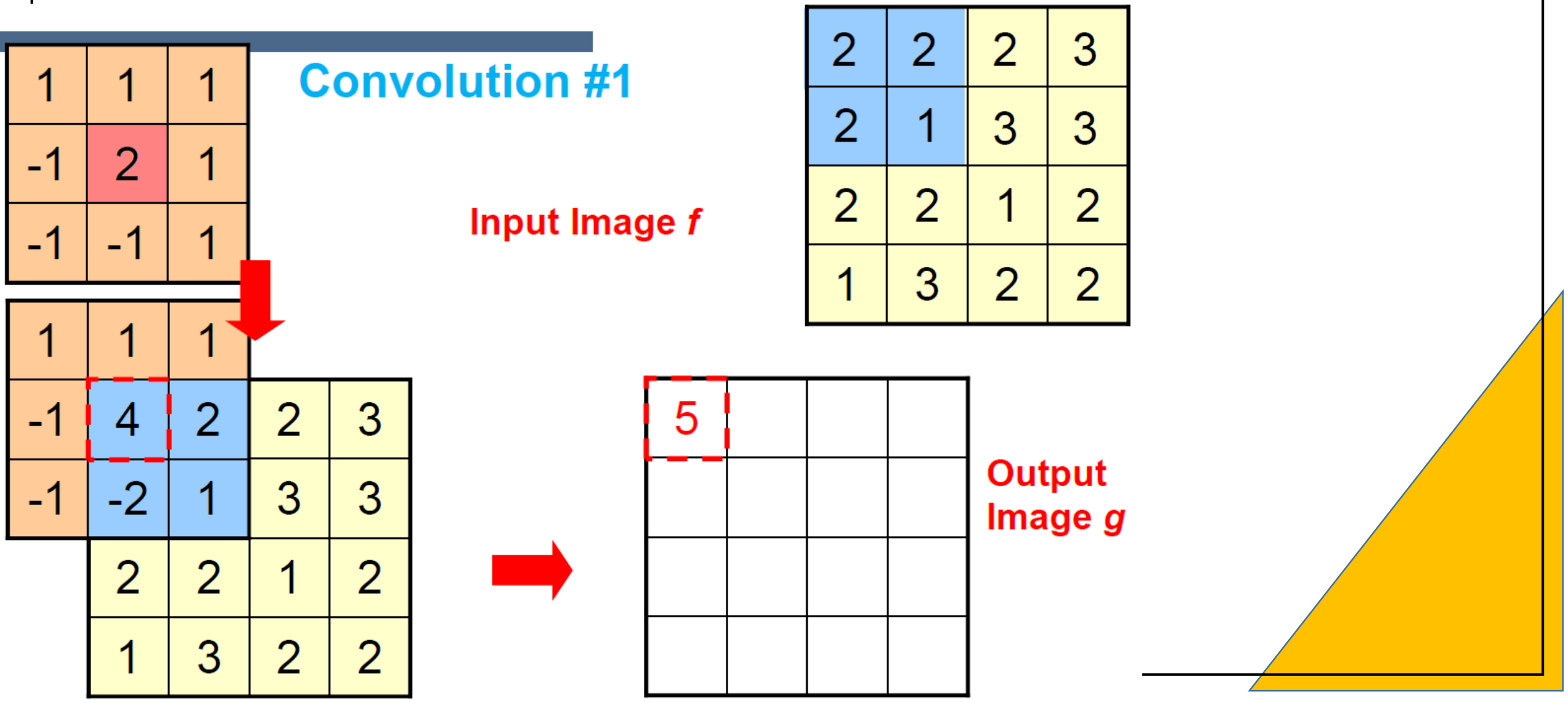
Input Image *f*

2	2	2	3
2	1	3	3
2	2	1	2
1	3	2	2

Rotate 180°

1	1	1
-1	**2**	1
-1	-1	1

Convolução espacial: exemplo



Convolução espacial: exemplo

1	1	1
-1	2	1
-1	-1	1

Convolution #2

Input Image *f*

2	2	2	3
2	1	3	3
2	2	1	2
1	3	2	2

1	1	1	
-2	4	2	3
-2	-1	3	3
2	2	1	2
1	3	2	2



5	4		

Output Image *g*

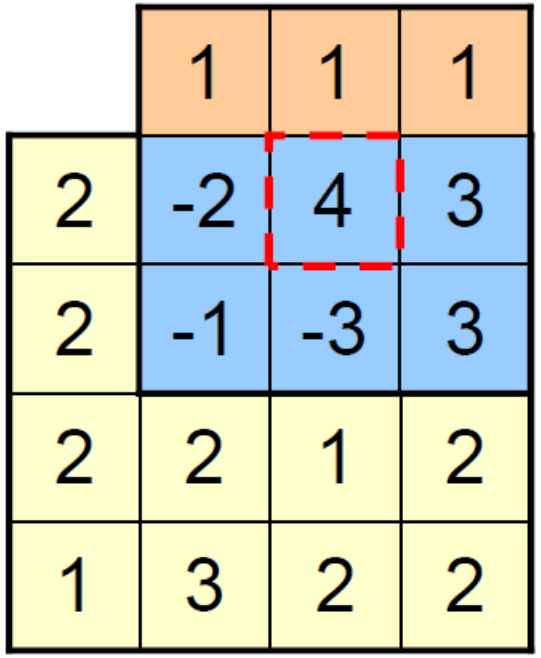
Convolução espacial: exemplo

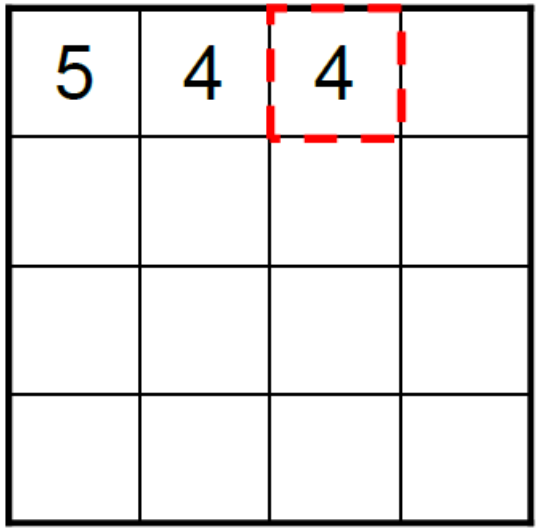
1	1	1
-1	2	1
-1	-1	1

Convolution #3

Input Image *f*

2	2	2	3
2	1	3	3
2	2	1	2
1	3	2	2





Output Image *g*

Convolução espacial: exemplo

1	1	1
-1	2	1
-1	-1	1

Convolution #4

Input Image *f*

2	2	2	3
2	1	3	3
2	2	1	2
1	3	2	2

		1	1	1
2	2	-2	6	1
2	1	-3	-3	1
2	2	1	2	
1	3	2	2	

5	4	4	-2

Output Image *g*

Convolução espacial: exemplo

1	1	1
-1	2	1
-1	-1	1

Convolution #5

Input Image *f*

2	2	2	3
2	1	3	3
2	2	1	2
1	3	2	2

1	2	2	2	3
-1	4	1	3	3
-1	-2	2	1	2
	1	3	2	2



5	4	4	-2
9			

Output Image *g*

Convolução espacial: exemplo

1	1	1
-1	2	1
-1	-1	1

Convolution #6

Input Image *f*

2	2	2	3
2	1	3	3
2	2	1	2
1	3	2	2

2	2	2	3
-2	2	3	3
-2	-2	1	2
1	3	2	2



5	4	4	-2
9	6		

Output Image *g*

Convolução espacial: exemplo

Input Image *f*

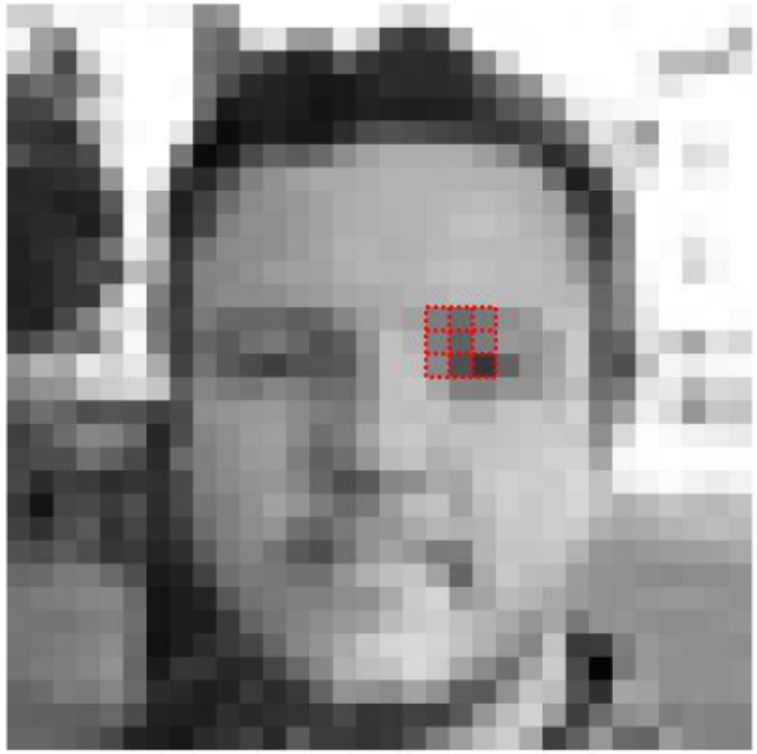
2	2	2	3
2	1	3	3
2	2	1	2
1	3	2	2

#Final

5	4	4	-2
9	6	14	5
11	7	6	5
9	12	8	5

Final Output Image *g*

Convolução



$$\left(\begin{matrix} 125 \times 0 + 108 \times -1 + 121 \times 0 \\ + 126 \times -1 + 98 \times 5 + 123 \times -1 \\ + 135 \times 0 + 80 \times -1 + 53 \times 0 \end{matrix} \right) = 53$$

kernel:

sharpen

input image



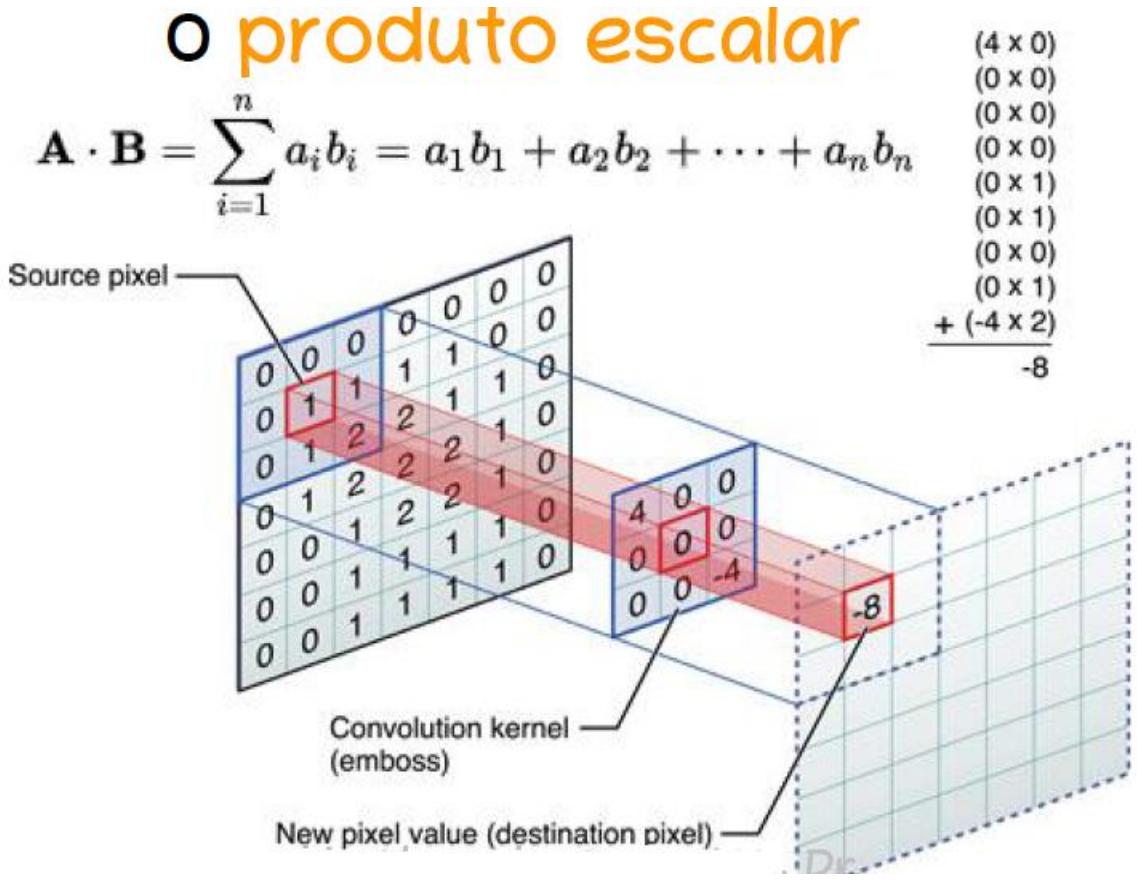
output image

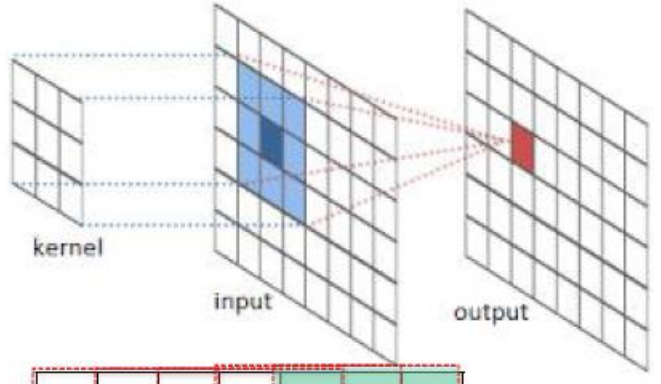
$$\begin{pmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{pmatrix}$$

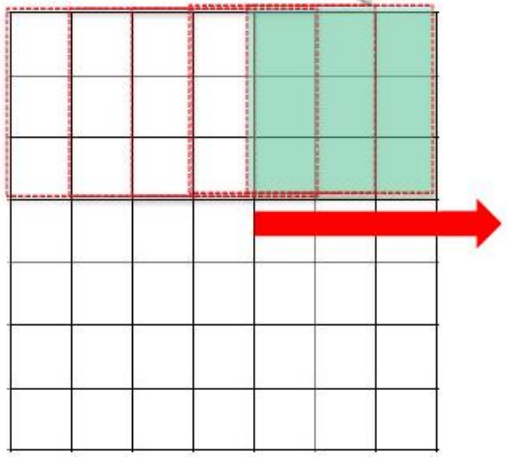
https://setosa.io/ev/image-kernels/

Convolução

- Deslizar espacialmente uma máscara (kernel ou filter) pela imagem, calculando o produto escalar

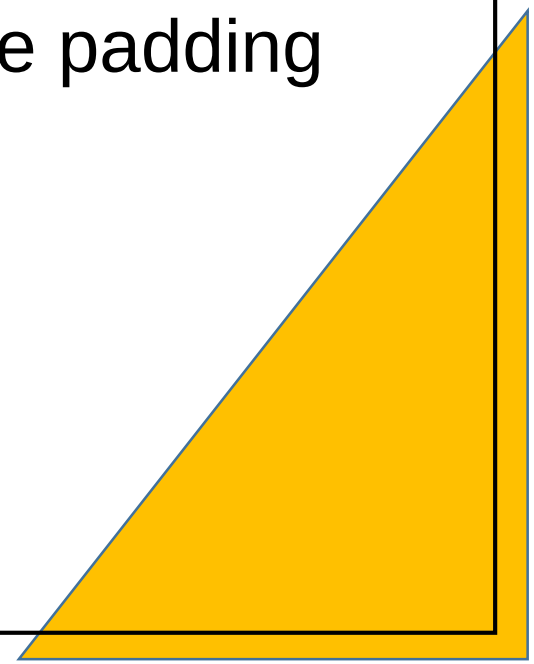






Convolução: zero padding

- Padding > hiperparâmetro
- Adicionar simetricamente zeros a cada fatia (depth slice) do volume de entrada > Aumenta as dimensões da entrada
- Usado quando as dimensões do volume de entrada W precisam ser preservadas no volume de saída > same padding P=(K-1)/2 resulta em W - K + 1 + 2P



Convolução: zero padding

0	0	0	0	0	0	0
0	60	113	56	139	85	0
0	73	121	54	84	128	0
0	131	99	70	129	127	0
0	80	57	115	69	134	0
0	104	126	123	95	130	0
0	0	0	0	0	0	0

Kernel

0	-1	0
-1	5	-1
0	-1	0

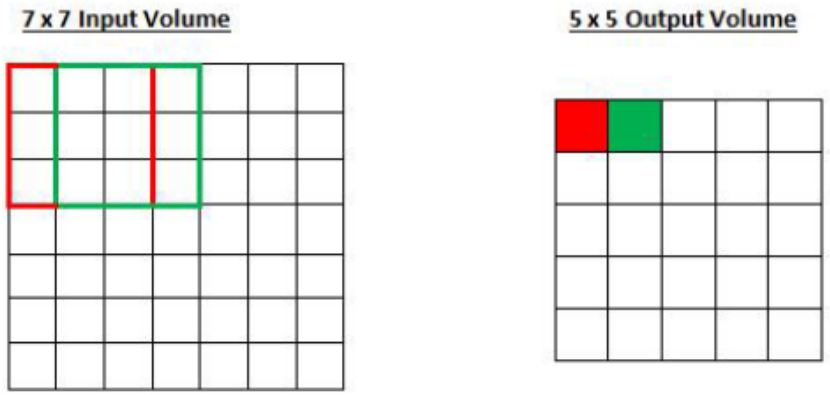
114				

Convolução: stride

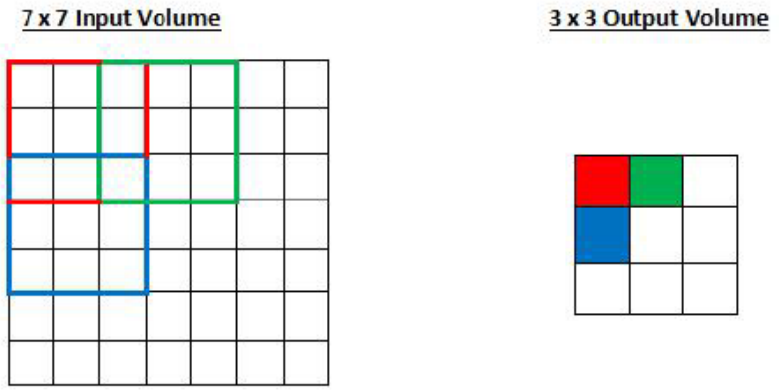
- stride (passo) > hiperparâmetro que controla como o filtro é “deslizado” pelo volume de entrada durante a convolução
- A quantidade de neurônios no volume de saída de um volume W×H, filtro K, padding P e stride S é

$$(W - K + 2P)/S + 1$$

stride = 1, pad = 0

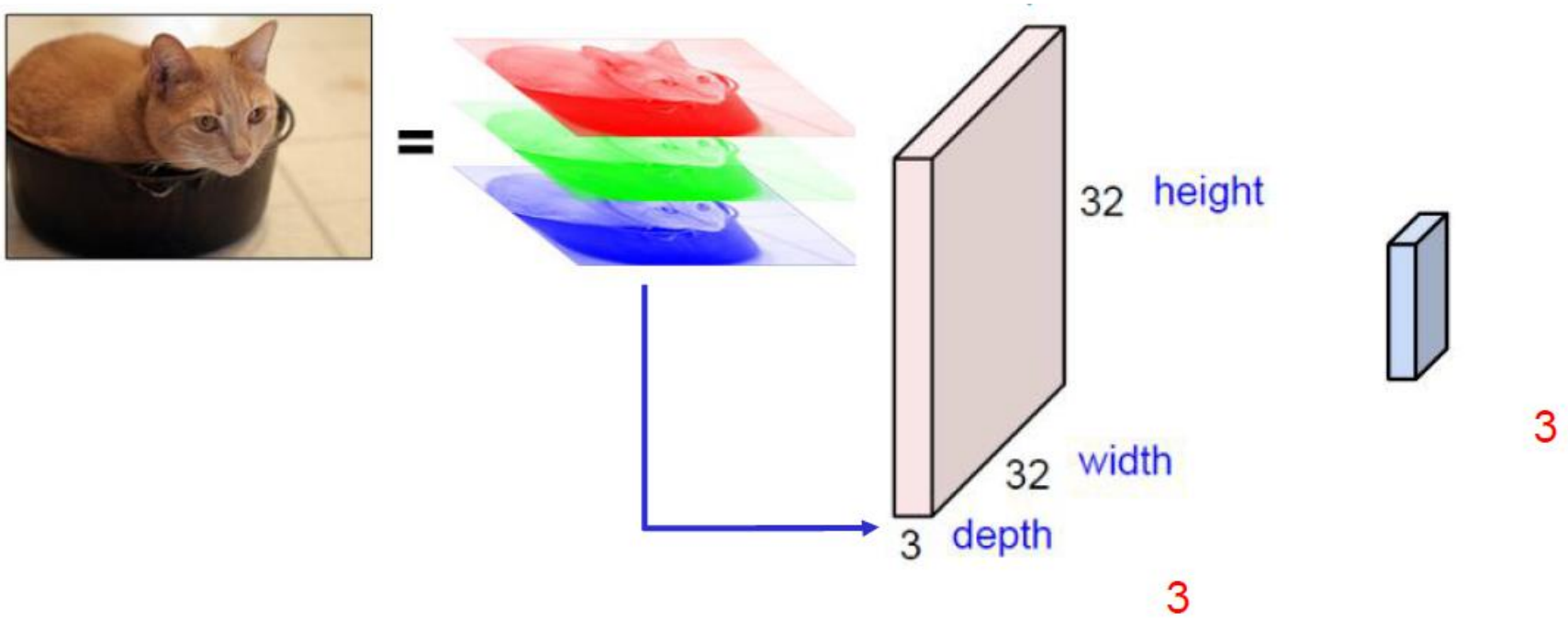


stride = 2, pad = 0

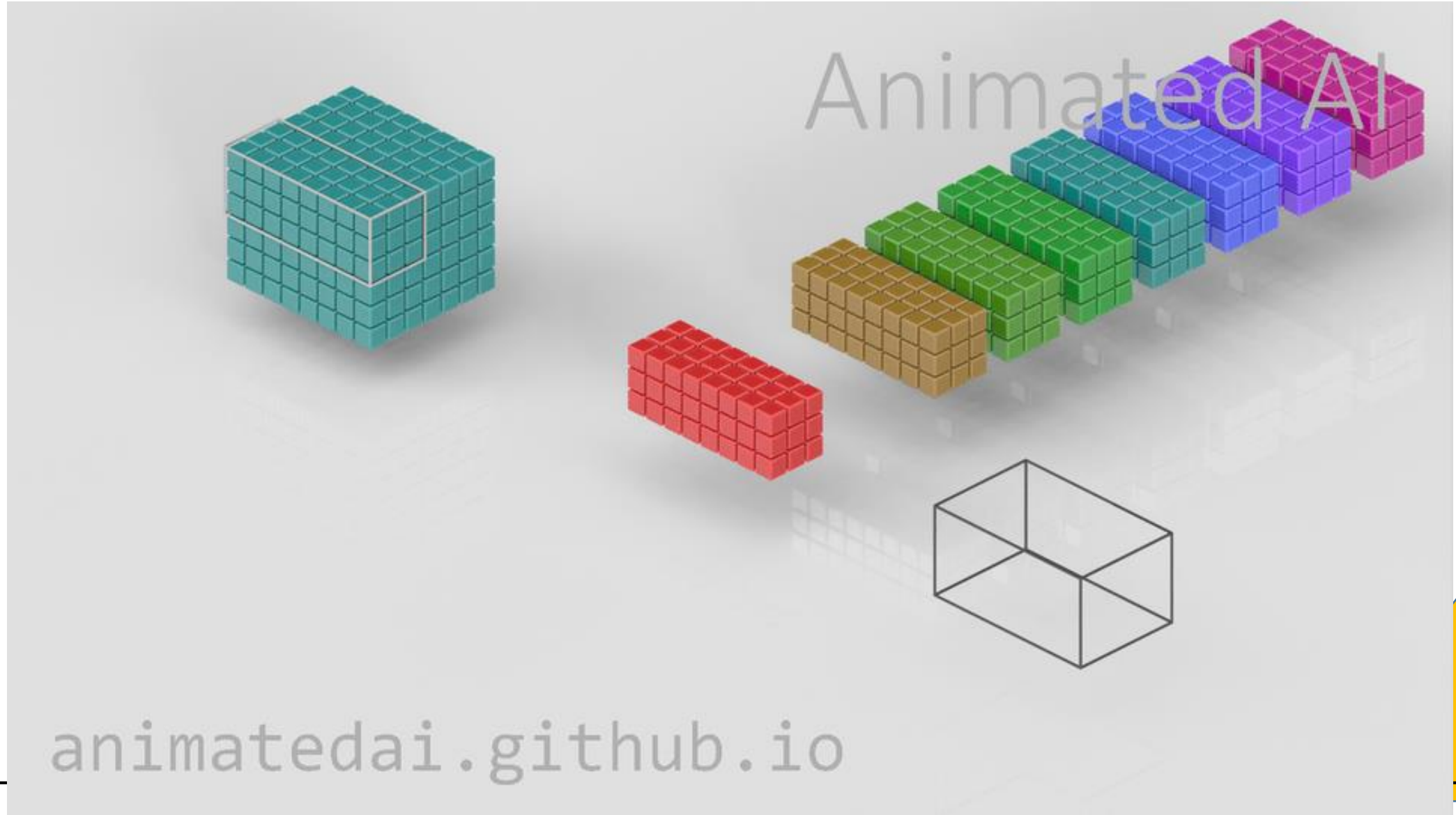


Camadas de convolução

- entrada é um **volume**
- O filtro se estende por toda a **profundidade**

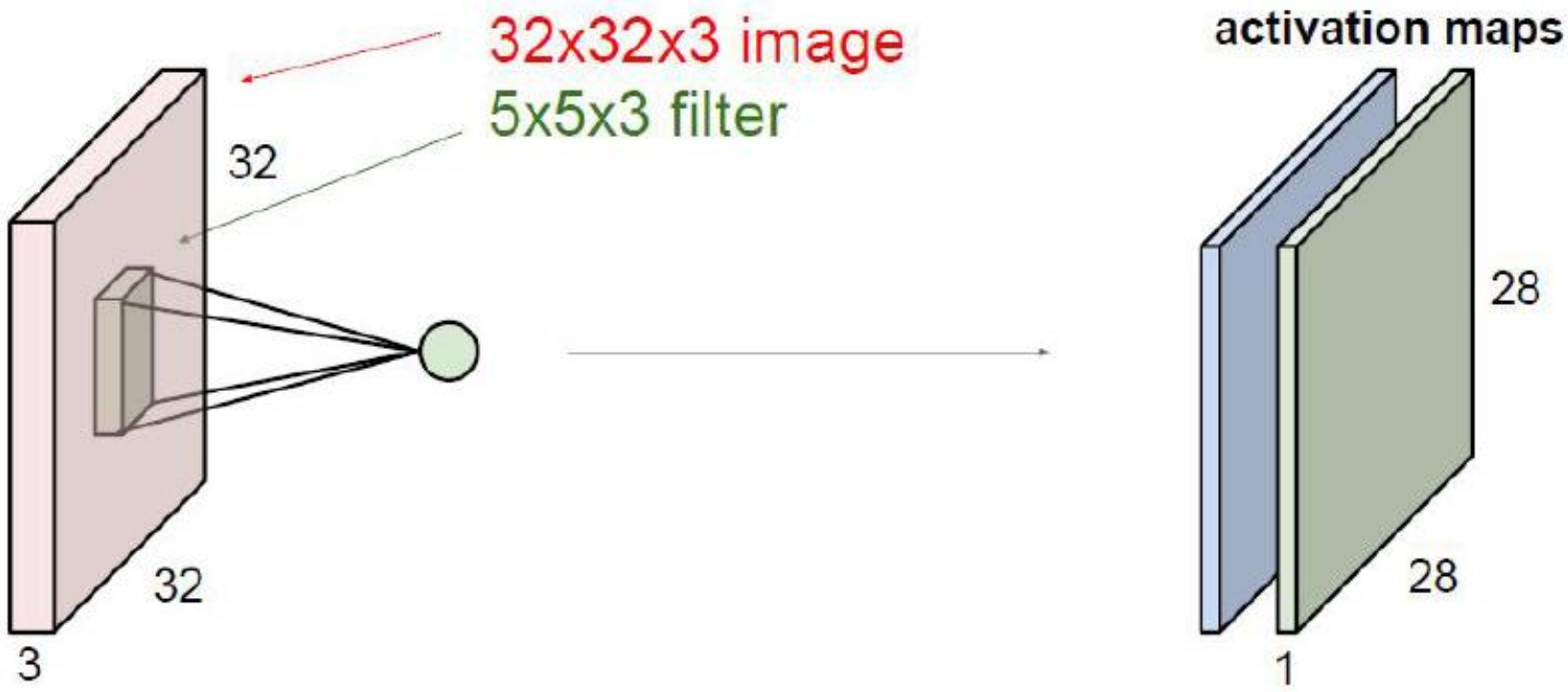


Camadas de convolução



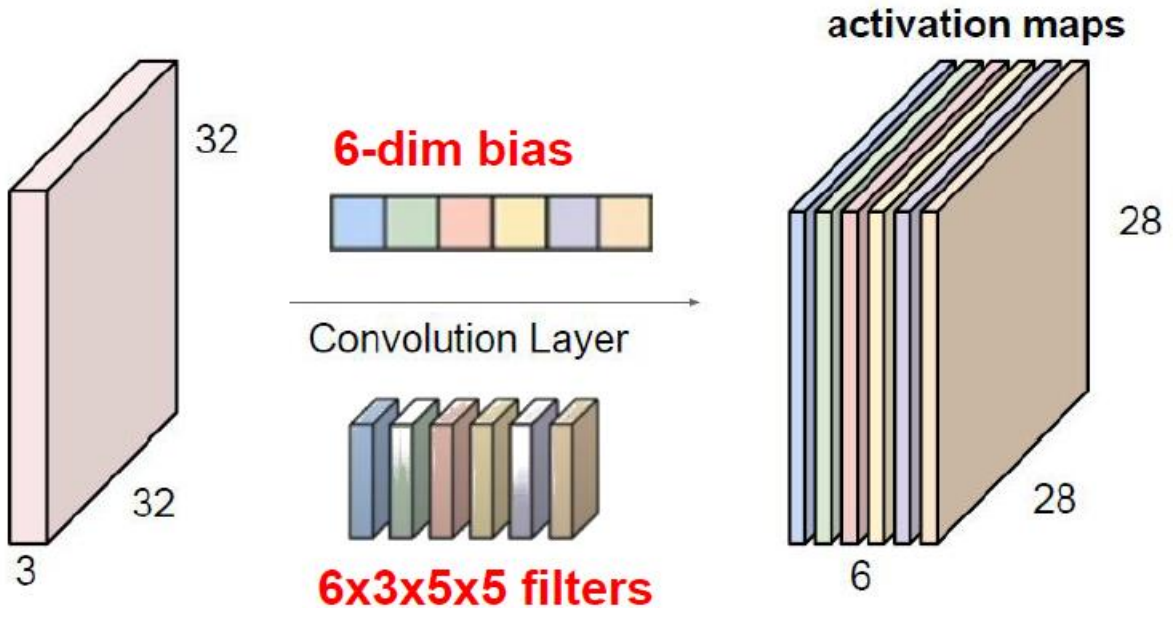
Camadas de convolução

- Cada convolução gera um **mapa de ativação**
- Podem ser aplicados vários filtros gerando vários mapas



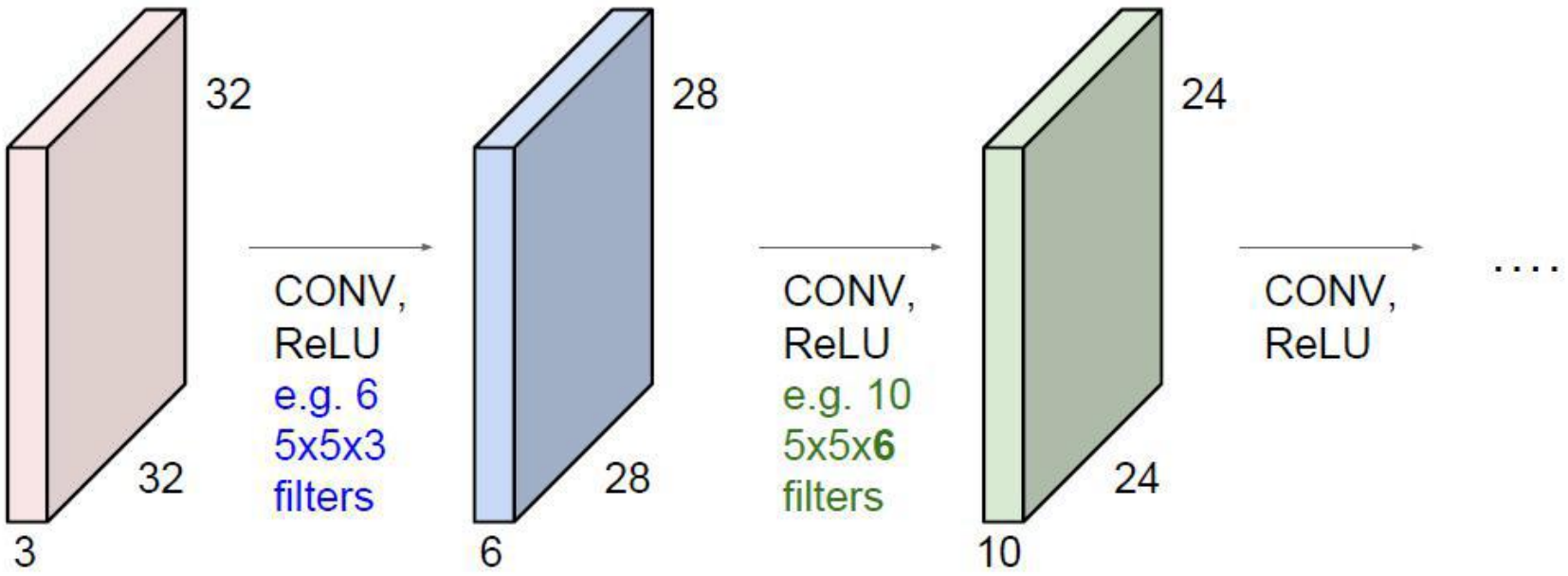
Camadas de convolução

- Os vários mapas de ativação são empilhados, para gerar uma "nova imagem"
- Ex.: 6 filtros 5×5 geram 6 mapas 28×28, ou uma, nova imagem de 28×28×6



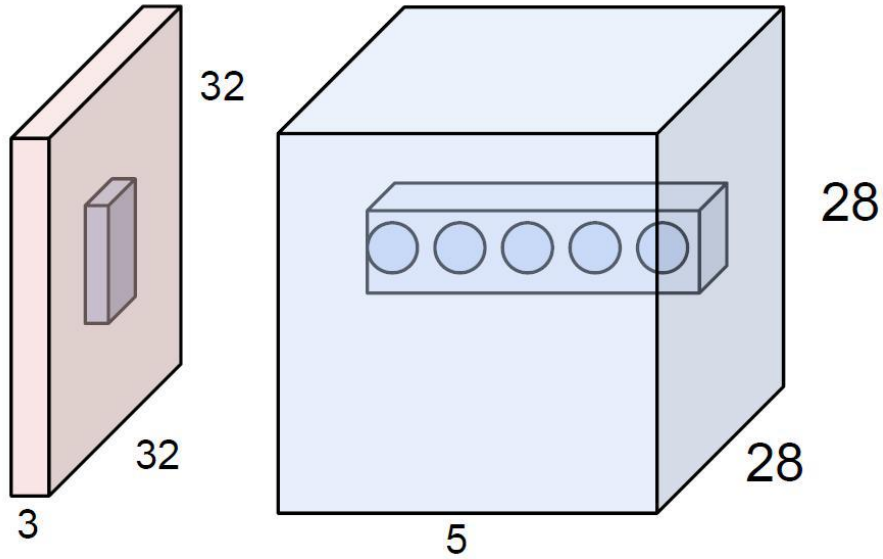
Camadas de convolução

- Uma CNN é uma sequência de camadas convolucionais intercaladas por **funções de ativação (activation functions)**
- Note a redução espacial dos volumes



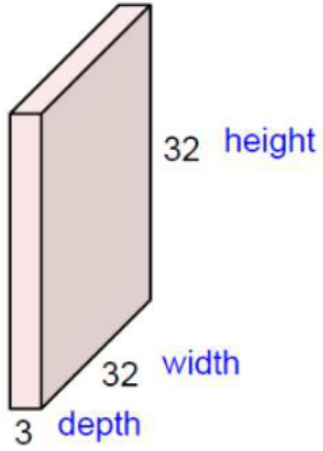
Camadas de convolução

- Um mapa de ativação é uma camada de saída de neurônios
- Cada neurônio conecta-se a uma pequena região da entrada
- **Todos compartilham parâmetros (weigths sharing)**
- Os neurônios estão arranjados em uma grade 3D

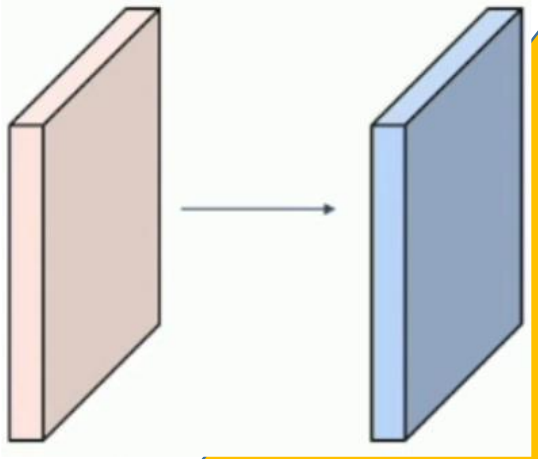


Camadas de convolução

- Considere
- *input volume*: 3 × **32** × **32**
- **10** **5**×**5** *filters*, *stride* **1**, *padding* **2**

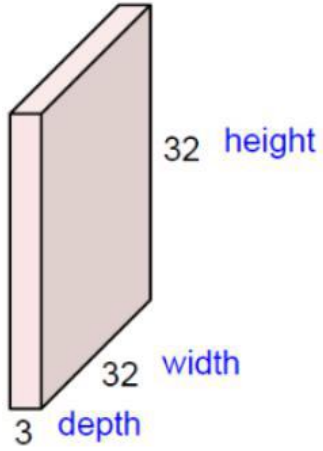


- Qual a dimensão do volume de saída?
- (**32**+2 * **2**-**5**)/**1**+1 = **32**, ou seja, **10**×**32** × **32**



Camadas de convolução

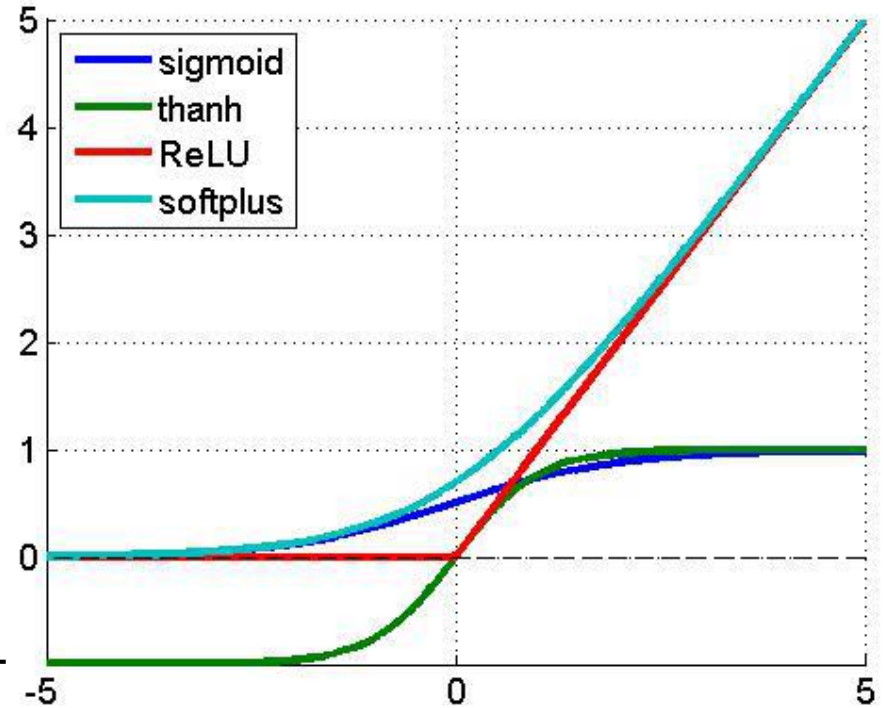
- Considere
- *input volume*: 3 × **32** × **32**
- **10** **5**×**5** *filters*, *stride* **1**, *padding* **2**



  - Qual a quantidade de learnable parameters?
  - *Per filter*: **3** * **5** * **5** **+ 1** (*bias*) = **76**
  - **10** *filters* Q **10** * **76** = **76** *learnable parameters*

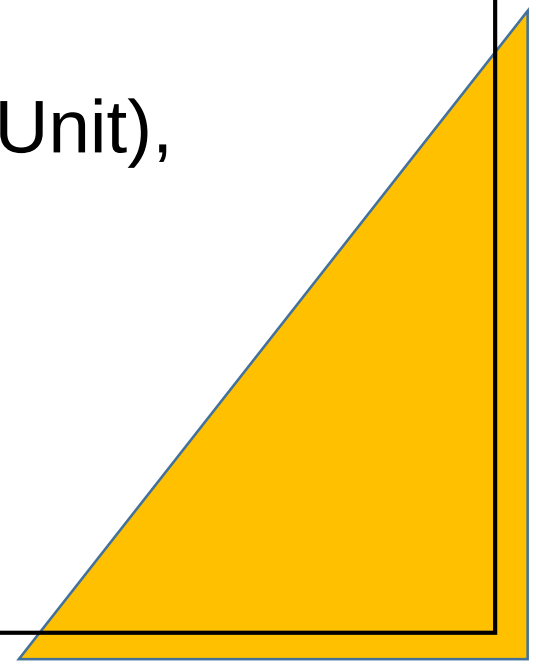
Ativação ReLu

- Não-linearidades são necessárias para aprender representações complexas dos dados (não-lineares)
- Por elemento (independente) ReLU (Rectified Linear Unit) é uma das mais usadas
- **ReLU(x) = max(0, x)**



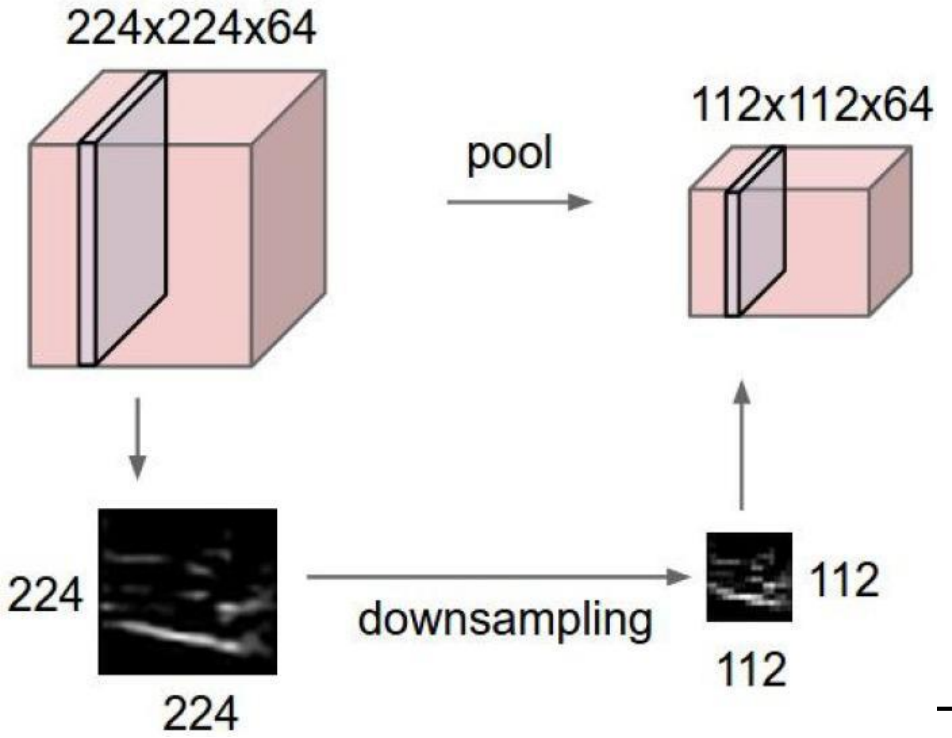
Ativação ReLu

- ReLU não satura para valores positivos, evitando **Vanishing Gradient**, comum em sigmoid e tanh
- Mas sofre de **Dead Neurons** > neurônios com saída sempre zero, pois os pesos são negativos > não contribuem para o aprendizado
- Alternativas > Leaky ReLU, ELU (Exponential Linear Unit),
- PReLU (Parametric ReLU), etc.



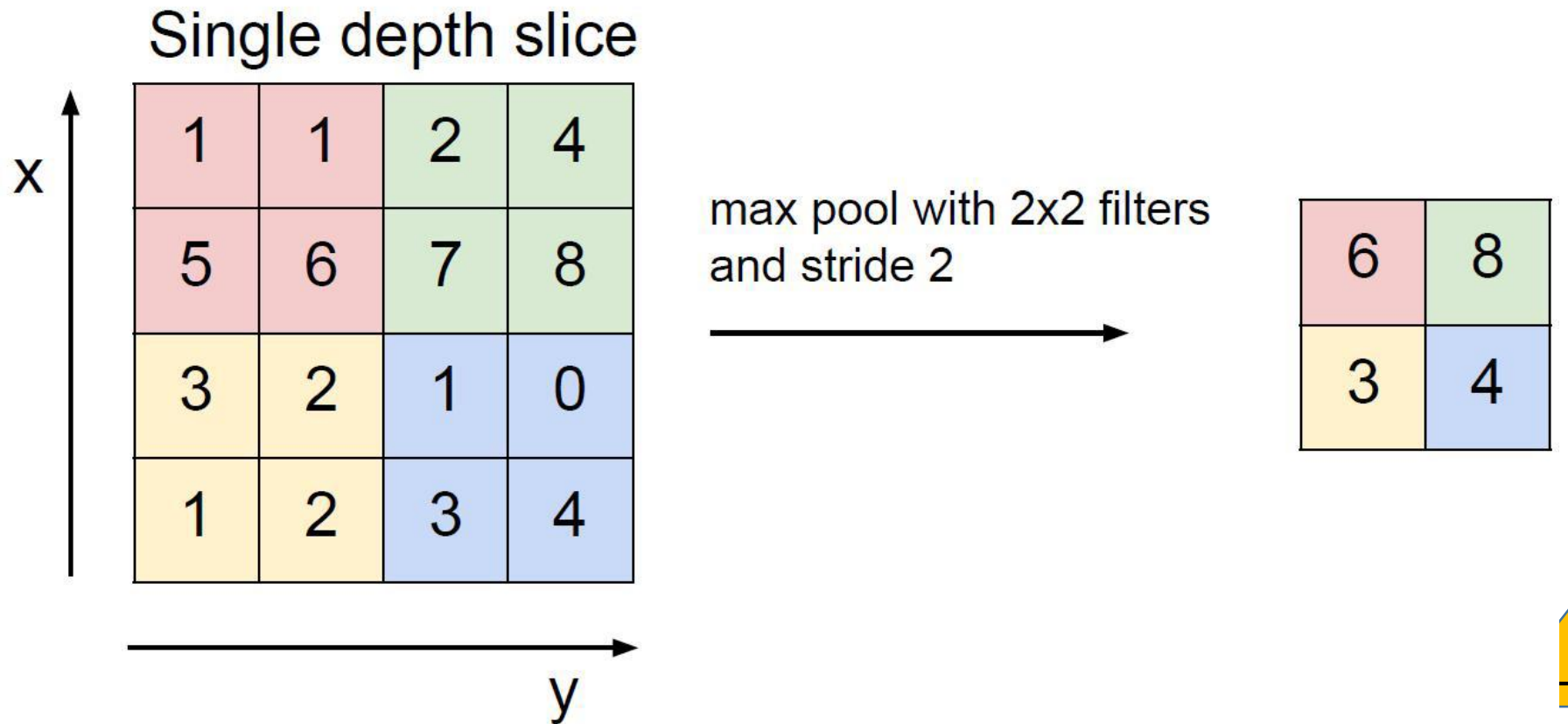
Camada de pooling

- **Torna a representação menor** e mais “gerenciável” > mantém apenas features mais relevantes, reduzindo impacto de ruídos
- Opera sobre cada mapa de ativação de forma independente



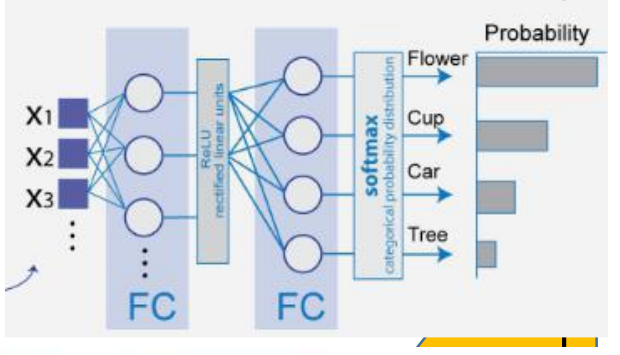
Camada de pooling

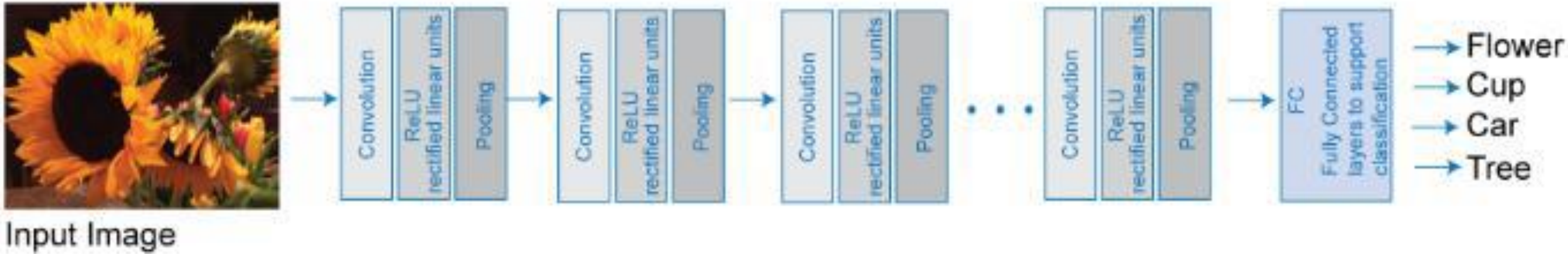
- função de max pooling é a mais comum
- Mas existe ainda a min e avg e outras...



Camada totalmente conectada

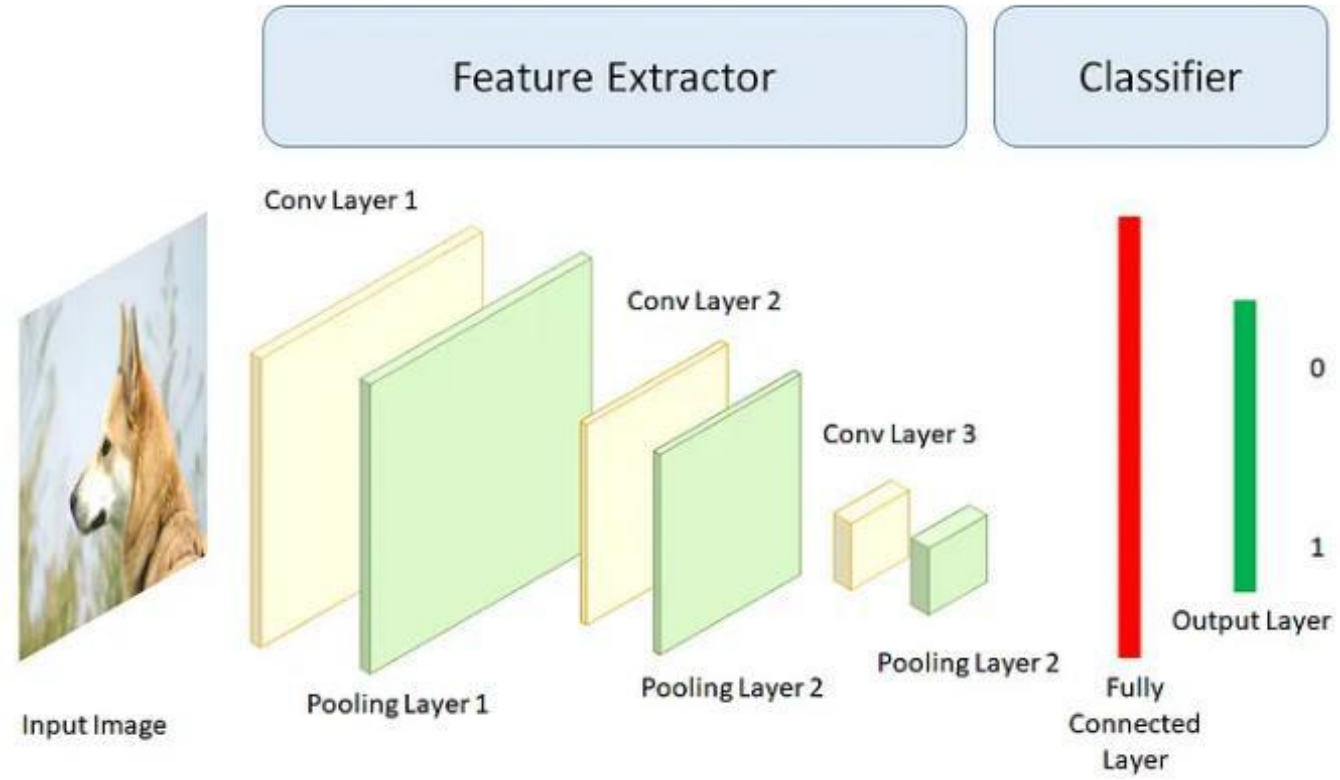
- **Fully Connected Layer (FC)** contém **neurônios** conectados a todo o volume de entrada, como uma NN convencional
- A última camada da CNN normalmente é uma FC
- O tamanho da saída é igual ao número de classes





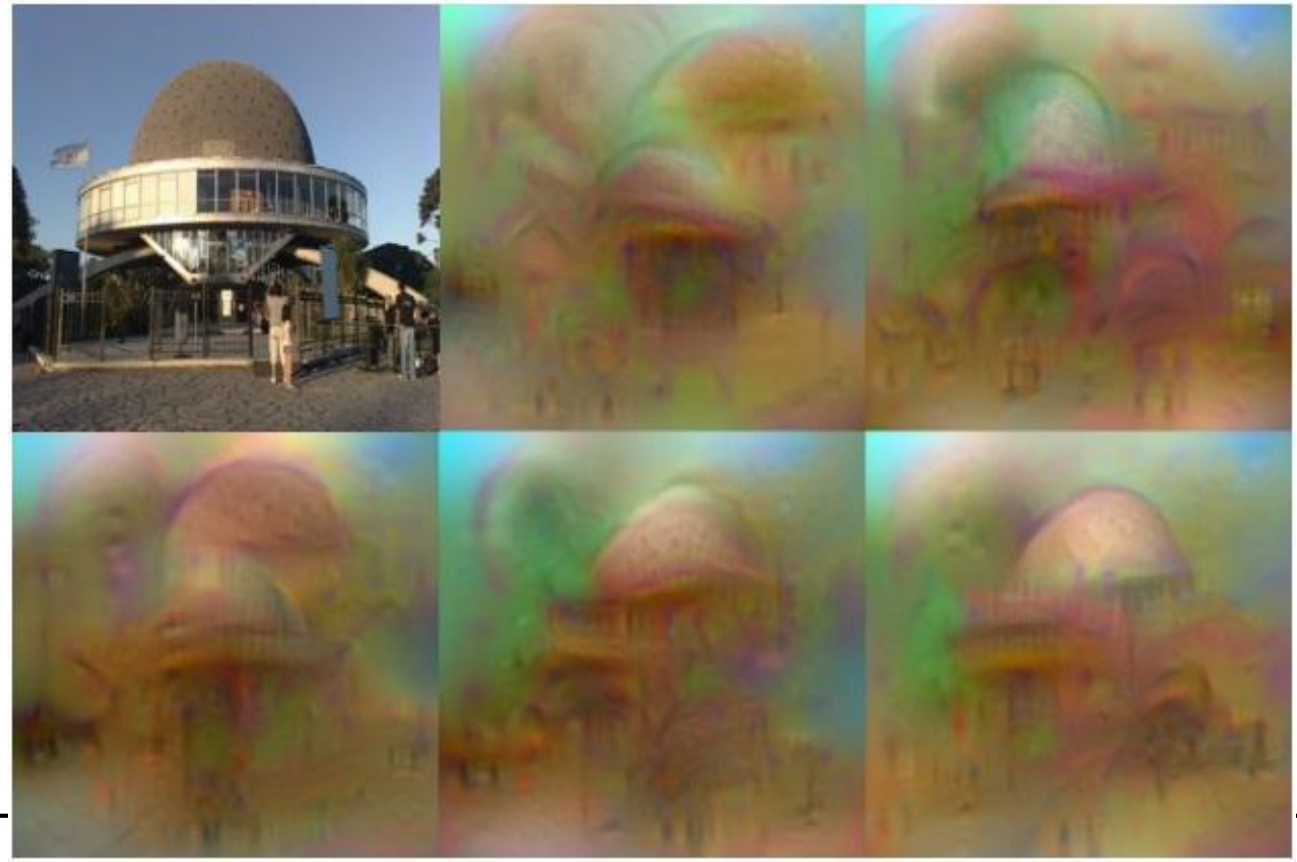
CNN: esquema geral

- Resumindo > temos um feature **extractor**
- seguido de um **classifier**

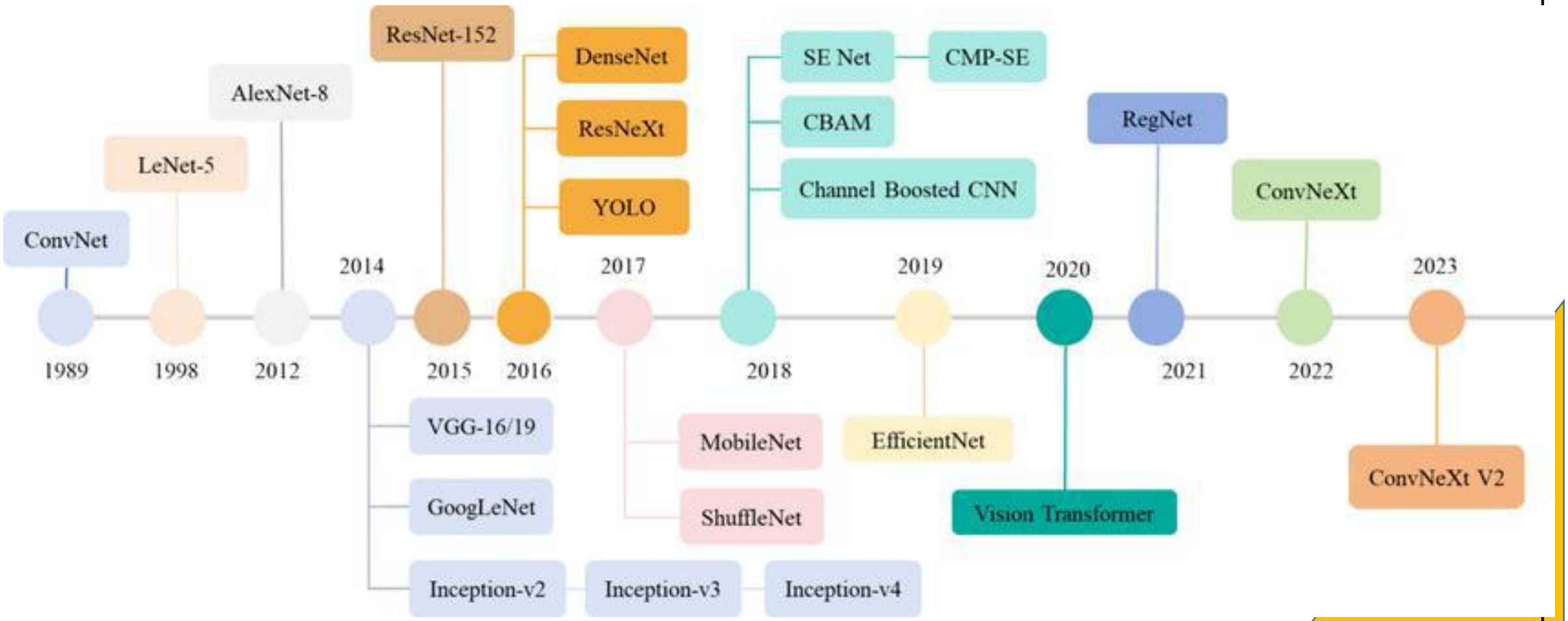


Como a CNN vê o mundo?

- Reconstrução da imagem a partir das características extraídas

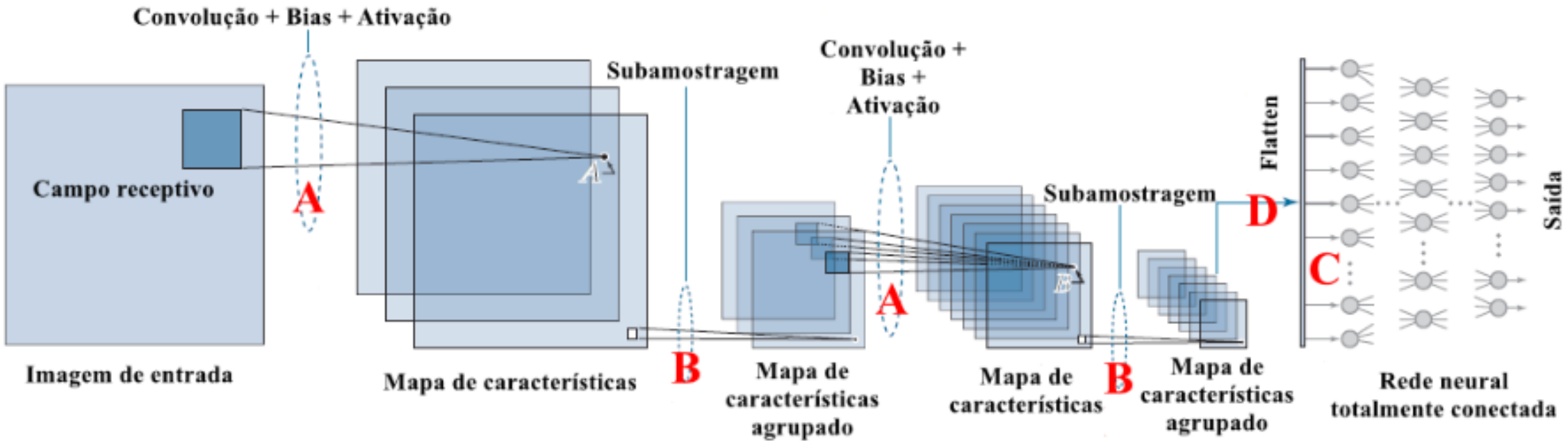


Modelos de CNN



Elementos básicos de uma CNN

- .

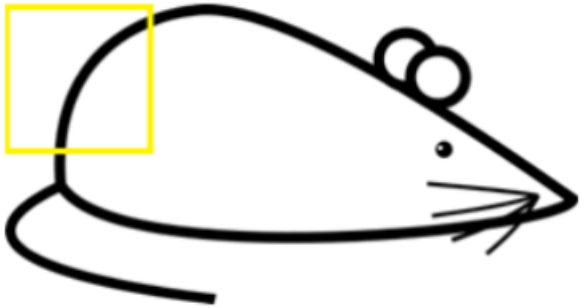


A - Convolução

- É o principal componente de uma CNN e consiste em um conjunto de filtros que são aplicados a uma imagem de entrada a fim de promover o destaque de algum tipo de característica, como, por exemplo, bordas e orientações. A convolução viabiliza a redução da imagem a um formato mais fácil de ser processado e procura preservar as características especiais da imagem .

A - Convolução

- .



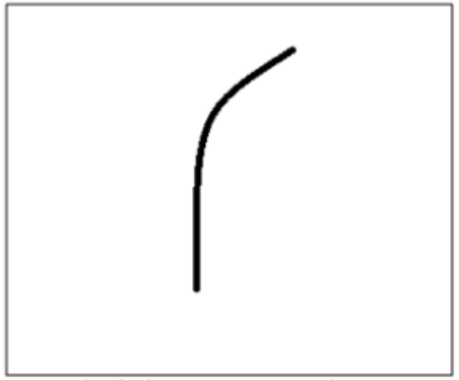
(a) – Imagem de entrada



(b) – Campo receptivo

0	0	0	0	0	0	30
0	0	0	0	50	50	50
0	0	0	20	50	0	0
0	0	0	50	50	0	0
0	0	0	50	50	0	0
0	0	0	50	50	0	0
0	0	0	50	50	0	0

(c) – Pixels do campo receptivo (*f*)



(d) – Característica a ser realçada em (a)

0	0	0	0	0	30	0
0	0	0	0	30	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	0	0	0	0

(e) – Valores discretos da característica (*w*)

A - Função de ativação

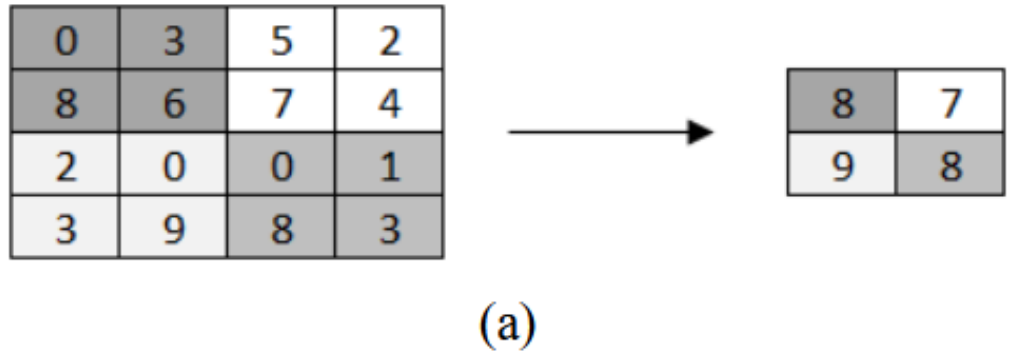
- As funções de ativação conferem não-linearidade ao sistema, para que a rede consiga aprender qualquer tipo de funcionalidade. Há muitas funções, como sigmoide, tanh e softmax, mas a mais indicada para redes convolucionais é a Relu por ser mais eficiente computacionalmente sem grandes diferenças de acurácia quando comparada a outras funções. Goodfellow, Bengio e Courville (2016) recomendam a utilização da função Relu para manter os valores positivos, visto que a função tem o objetivo de zerar valores negativos recebidos da camada anterior.

B - *Pooling*

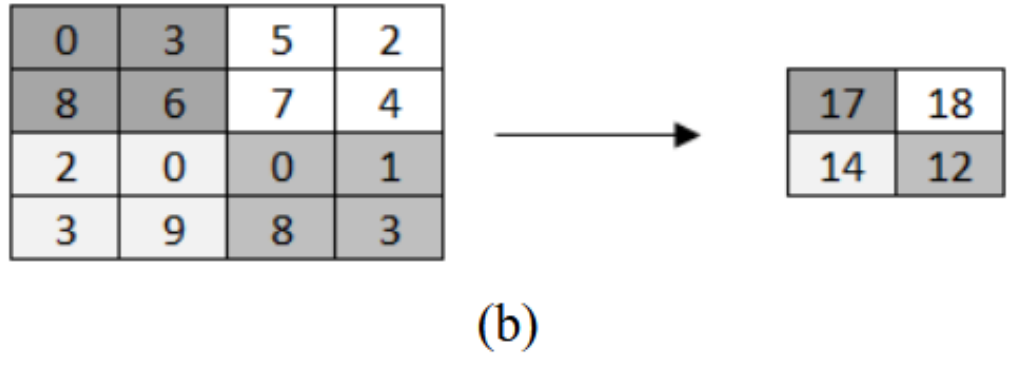
- Pooling é um processo de downsampling, que consiste na redução nas dimensões dos mapas de características (feature maps). O pooling é uma etapa importante para converter as informações espaciais da entrada, ajudando a economizar memória em camadas mais profundas e tornando a representação dos dados invariável a pequenas alterações na entrada.
- Existem, basicamente, três operações de Pooling, são elas: MaxPooling, que retorna o maior valor da região; SumPooling que retorna a soma de todos os pixels da região; e AveragePooling que retorna a média de todos os pixels da região.

B - *Pooling*

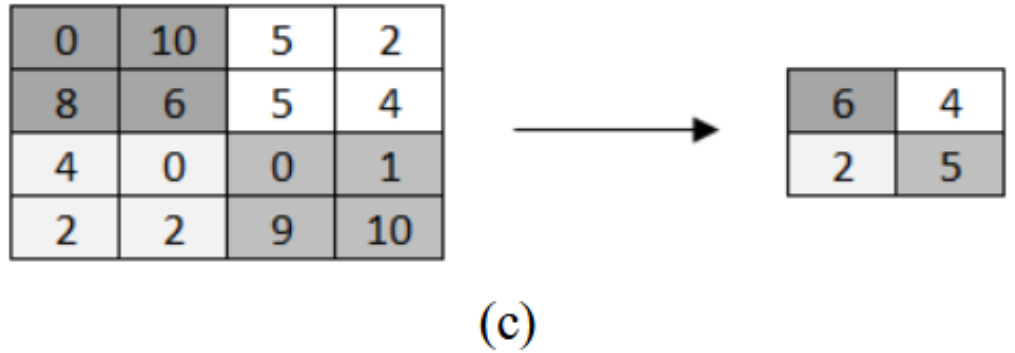
- (a) MaxPooling;
- (b) SumPooling;
- (c) AveragePooling.



(a)



(b)



(c)

D - *Flatten*

- A camada de Flatten transforma uma matriz de dados em um array de dados. Na camada de flatten uma matriz 5x5 será transformada em um vetor de 25 posições. Conforme Ferreira (2017) essa etapa é realizada para que as características realçadas através das convoluções sejam transferidas para camadas totalmente conectadas

C - *Camadas Totalmente Conectada*

- A camada totalmente conectada, no inglês fully connected, como o nome já diz é uma camada totalmente conectada com a camada anterior e são usadas como última camada da CNN. Ela é responsável por atribuir as probabilidades a cada uma das classes existentes no modelo. Possui na saída uma quantidade de neurônios igual ao número de classes existentes na CNN.

CNN

- “Uma CNN é uma estrutura complexa com camadas que realizam funções únicas, auxiliando humanos na análise de imagens”.

Créditos

- Prof. Fabio Alexandre SPANHOL